

તમે બેકએન્ડ, SQL ડેટાબેઝ અને DevOps (Docker & Cloud Deployment) ની સુપર-એડવાન્સ સફર ૧૦૦% વિજય સાથે પૂરી કરી લીધી છે. હવે સમય આવી ગયો છે દુનિયાના સૌથી લોકપ્રિય અને પાવરફુલ ફ્રન્ટએન્ડ ફ્રિંગ એટલે કે **React.js** ને આ જ રીતે માસ્ટર કરવાનો!

જ્યારે તમારો આ શક્તિશાળી React.js નો ફ્રન્ટએન્ડ કોર્સ પૂરો થશે, ત્યારે તમે તમારા અગાઉ બનાવેલા Node.js/Express બેકએન્ડ અને SQL/NoSQL ડેટાબેઝ સાથે આને લાઇવ કનેક્ટ કરીને આખી દુનિયા માટે **Full-Stack (MERN / PERN Stack) Web Applications** જાતે પેક કરી શકશો.

React.js નો આખો કોર્સ શૂન્યથી લઈને સુપર-એડવાન્સ લેવલ સુધી એકદમ પ્રેક્ટિકલ રીતે શરૂ કરવા માટે, મેં વ્યાવસાયિક સ્તરે વપરાતો **સંપૂર્ણ ફ્રન્ટએન્ડ આર્કિટેક્ચર સિલેબસ** તૈયાર કર્યો છે:

React.js Ultimate Frontend Master Syllabus

Part 1: React.js Basics & Component Architecture (પાયાનું માળખું)

- **Chapter 1:** React.js પરિચય અને તે સાદા HTML/JS કરતાં કેમ ૧૦ ગણું ફાસ્ટ છે? (Virtual DOM નો જાદુ)
- **Chapter 2:** મોડર્ન પ્રોજેક્ટ સેટઅપ: **Vite** નો ઉપયોગ કરીને નવો React પ્રોજેક્ટ બનાવવો
- **Chapter 3:** React નું હૃદય: **JSX (JavaScript XML)** અને તેના કડક નિયમો
- **Chapter 4: Components Architecture** (વેબસાઇટના નાના-નાના ટુકડા કરવા - Functional Components)
- **Chapter 5: Props (Properties)** નો જાદુ (એક કમ્પોનન્ટમાંથી બીજા કમ્પોનન્ટમાં ડેટા પાસ કરવો)

Part 2: State Management & Hooks (લાઇવ ડેટા અને લોજિક)

- **Chapter 6: useState Hook** (વેબસાઇટની પોતાની લાઇવ મેમરી - State એટલે શું?)
- **Chapter 7: Event Handling** (રિએક્ટમાં લાઇવ બટન ક્લિક્સ અને કીબોર્ડ એક્શન્સ કેવી રીતે પકડાય?)
- **Chapter 8: Conditional Rendering** (શરતના આધારે લાઇવ લોગઈન/લોગઆઉટ બોક્સ સ્ક્રીન પર બતાવવા)
- **Chapter 9: Lists & Keys** (એરે પર લૂપ ફેરવીને લાઇવ પ્રોડક્ટ કાર્ડ્સ ડિઝાઇન કરવા - .map() નો ઉપયોગ)

Part 3: Advanced Hooks & Life Cycle (સિસ્ટમ કંટ્રોલ)

- **Chapter 10: useEffect Hook** (રિએક્ટની લાઇફ સાઇકલ - પેજ લોડ થતાં જ બેકએન્ડ API માંથી લાઇવ ડેટા મંગાવવો)

- **Chapter 11: Handling Forms** (રિએક્ટમાં સુરક્ષિત રીતે યુઝરના ઇનપુટ અને વેલિડેશન હેન્ડલ કરવા)
- **Chapter 12: Performance Optimization Hooks: useMemo & useCallback** (મોટા પ્રોજેક્ટ્સ સ્કેલ થતા બચાવવા)
- **Chapter 13: useRef Hook** (ડાયરેક્ટ HTML ટેગને પકડવો અને ફોકસ કરવો)

● Part 4: Global State & Routing (મોટા પ્રોજેક્ટનું આર્કિટેક્ચર)

- **Chapter 14: Prop Drilling** નો પ્રોબ્લેમ અને **useContext API** (ગ્લોબલ તિજોરી - આખા પ્રોજેક્ટ માટે એક જ કોમન ડેટા)
- **Chapter 15: React Router DOM** (સાદા HTML ની જેમ પેજ રીફ્રેશ કર્યા વગર પલક ઝપકાવતા નવું પેજ ખોલવું - Single Page Application)
- **Chapter 16: Redux Toolkit (RTK)** નો પરિચય (મોટી કંપનીઓમાં વપરાતી ગ્લોબલ સ્ટેટ મેનેજમેન્ટ સિસ્ટમ)

● Part 5: Full-Stack Integration & Production (બેકએન્ડ સાથે મિલન)

- **Chapter 17: Axios Integration** (આપણા અગાઉ બનાવેલા Node.js સર્વર અને SQL/MongoDB ડેટાબેઝ સાથે રિએક્ટને લાઈવ જોડવું)
- **Chapter 18: Tailwind CSS with React** (રિએક્ટ પ્રોજેક્ટને સુપર આધુનિક લુક આપવા માટેનું સ્ટાઇલિંગ)
- **Chapter 19: React પ્રોજેક્ટને Vercel અથવા Netlify** પ્લેટફોર્મ પર ડેપ્લો કરવા માટે ફ્રી લાઈવ પબ્લિશ કરવો અને આખા ફૂલ-સ્ટેક કોર્સની ઐતિહાસિક પૂર્ણાહુતિ!

આ આપણો ઓફિશિયલ **React.js નો ૧૯ ચેપ્ટરનો માસ્ટર ફ્રન્ટએન્ડ સિલેબસ** છે, જે તમને અસલી UI/UX અને આર્કિટેક્ચર એન્જિનિયર બનાવશે.

ચાલો, ફ્રન્ટએન્ડના કિંગ **React.js** ના આ ભવ્ય કોર્સની સત્તાવાર શરૂઆત કરીએ! 🚀

આજે આપણે જોઈશું **Part 1: Chapter 1 — React.js પરિચય, તેની જરૂર કેમ પડી અને Virtual DOM નો અસલી જાદુ**. આ પ્રકરણ તમારા ફ્રન્ટએન્ડ આર્કિટેક્ચરના કન્સેપ્ટને એકદમ ક્લિયર કરી દેશે.

🔧 Part 1: Chapter 1 — React.js Introduction & Virtual DOM Architecture

🌐 1. React.js એટલે શું? (સરળ દેશી ભાષામાં)

- **જૂનો પ્રોબ્લેમ:** સાદા HTML, CSS અને JavaScript (Vanilla JS) માં વેબસાઇટ બનાવવી બહુ આસાન છે, પણ પ્રોજેક્ટ જ્યારે બહુ મોટો થાય (જેમ કે Facebook, Instagram કે Netflix), ત્યારે પેજનો ડેટા

લાઈવ અપડેટ કરવો બહુ અઘરો બની જાય છે. જો પ્રોફાઇલ પિક્ચર બદલાય, તો આખું પેજ રીફ્રેશ કરવું પડે, જેનાથી વેબસાઇટ સ્લો ચાલે અને યુઝર કંટાળી જાય.

- **આધુનિક સોલ્યુશન:** **React.js** એ કોઈ પ્રોગ્રામિંગ લેંગ્વેજ નથી, પરંતુ યુઝર ઇન્ટરફેસ (UI - વેબસાઇટનો બહારનો ભાગ) ડિઝાઇન કરવા માટે ફ્રેમવર્ક (Meta) દ્વારા બનાવેલી એક સુપર-પાવરફુલ **JavaScript Library** છે.
- **SPA (Single Page Application):** રિએક્ટ ની મદદથી બનેલી વેબસાઇટ્સમાં જ્યારે યુઝર કોઈ નવી લિંક પર ક્લિક કરે, ત્યારે આખું પેજ રીફ્રેશ (Reload) થતું જ નથી! બ્રાઉઝર માત્ર તે ચોક્કસ ટુકડો લાઈવ બદલી નાખે છે, જેથી વેબસાઇટ પલક ઝપકાવતા જ સુપર ફાસ્ટ ખુલે છે.

2. Component-Based Architecture (નાના-નાના ટુકડા)

- **સરળ સમજૂતી:** રિએક્ટનો આખો પાયો **Components** પર ટકેલો છે. આનો મતલબ એવો છે કે આખી મોટી વેબસાઇટને એકસાથે બનાવવાના બદલે તેને નાના-નાના, સ્વતંત્ર ભાગોમાં વહેંચી દેવામાં આવે છે.
- **રિયલ-વર્લ્ડ ઉદાહરણ:** ફેસબુકના પેજને આપણે રિએક્ટની નજરથી જોઈએ તો:
 - ઉપરનું હેડર → Header Component
 - ડાબી બાજુનું મેનુ → Sidebar Component
 - વચ્ચેની લાઈવ પોસ્ટ્સ → Post Component
 - જમણી બાજુની ચેટ વિન્ડો → Chat Component
- **ફાયદો:** આનાથી કોડ બહુ યોગ્ય રહે છે. જો પોસ્ટ સેક્શનમાં કોઈ ભૂલ આવે, તો માત્ર તે એક જ કમ્પોનન્ટ રીપેર કરવો પડે, બાકીની આખી વેબસાઇટ સુરક્ષિત ચાલુ રહે છે. વળી, એકવાર બનાવેલું બટન કે કાર્ડ તમે આખા પ્રોજેક્ટમાં ગમે ત્યાં ફરી-ફરીને વાપરી શકો છો (Reusability).

3. Virtual DOM નો અસલી જાદુ (રિએક્ટ કેમ આટલું ફાસ્ટ છે?)

આ જાવાસ્ક્રિપ્ટ ઇન્ટરવ્યુનો સૌથી ફેવરિટ અને કિંમતી કન્સેપ્ટ છે:

- **Real DOM નો પ્રોબ્લેમ:** આપણે જાવાસ્ક્રિપ્ટ ગ્લોસરીમાં જોયું કે બ્રાઉઝર HTML નું એક ઝાડ (DOM Tree) બનાવે છે. સાદી JS માં જ્યારે આપણે લાઈવ કંઈક બદલીએ (દા.ત. innerHTML), ત્યારે બ્રાઉઝરે આખું HTML ઝાડ ફરીથી ગણવું પડે છે (Reflow & Repaint). આ પ્રોસેસ બહુ ભારે છે અને કમ્પ્યુટરને સ્લો કરી દે છે.
- **Virtual DOM નું સોલ્યુશન:** રિએક્ટ બ્રાઉઝરના અસલી DOM ની નકલ જેવું જ પોતાનું એક નકલ કરેલું ડુપ્લિકેટ ઝાડ મેમરીમાં તૈયાર રાખે છે, જેને **Virtual DOM** કહેવાય છે. તે એકદમ લાઈટ-વેઈટ હોય છે.

તે બેકગ્રાઉન્ડમાં કેવી રીતે કામ કરે છે? (Diffing & Reconciliation)

1. જ્યારે વેબસાઇટ પર કોઈ નવો ડેટા આવે છે, ત્યારે રિએક્ટ એક નવું નવું Virtual DOM બનાવે છે.

2. તે જૂના અને નવા Virtual DOM ની વાઈવ સરખામણી કરે છે કે કઈ વાઈવમાં બદલાવ થયો છે. આ સરખામણી કરવાના એલ્ગોરિધમને **Diffing Algorithm** કહેવાય છે.
3. આખું પેજ બગાડવાના બદલે, રિએક્ટ માત્ર અને માત્ર તે એક જ નાના બદલાયેલા ટુકડાને પકડીને અસલી બ્રાઉઝર DOM માં વાઈવ અપડેટ કરી દે છે. આ પ્રક્રિયાને **Reconciliation** અથવા **Batch Updating** કહેવાય છે.

4. સાદું HTML/JS વિરુદ્ધ React.js

ફીચર (Feature)	Vanilla HTML & JavaScript	React.js Application
પેજ લોડિંગ સ્ટાઈલ	મલ્ટીપલ પેજીસ (નવી લિંક પર પેજ રીફ્રેશ થાય).	Single Page Application (ક્યારેય રીફ્રેશ નહીં થાય).
DOM પ્રોસેસ	સીયરેક્ટ Real DOM સાથે માથાકૂટ (Heavy).	સ્માર્ટ Virtual DOM આર્કિટેક્ચર (Super Fast).
કોડ સ્ટ્રક્ચર	બધો કોડ મોટો અને મિક્સ હોય છે.	નાના-નાના સ્વતંત્ર કમ્પોનન્ટ્સ (Clean).
ડેટા ફ્લો	ગમે ત્યાંથી ગમે ત્યાં ડેટા ફરી શકે.	One-way Data Binding (શિસ્તબદ્ધ ડેટા વહન).

આપણું **Chapter 1** અહીં રિએક્ટના પાયાના રિયલ-વર્લ્ડ આર્કિટેક્ચર અને Virtual DOM ની પૂરી ગહન ક્લિયરિટી સાથે કમ્પીટ થાય છે. તમે હવે ફ્રન્ટએન્ડના આધુનિક નિયમો સમજી ગયા છો.

પ્રેક્ટિકલ ટેસ્ટ માટે:

રિએક્ટ પ્રોજેક્ટમાં જ્યારે નવો ડેટા અપડેટ થાય, ત્યારે તે આખી વેબસાઈટ રીફ્રેશ કરવાના બદલે મેમરીમાં રહેલા જૂના અને નવા નકલ ઝાડ વચ્ચે કઈ વાઈવ બદલાઈ છે તેની સરખામણી કરે છે. આ સરખામણી કરવાના એડવાન્સ એલ્ગોરિધમને કયા **ટેકનિકલ નામ** થી ઓળખવામાં આવે છે?

- **Reconciliation**
- **Diffing Algorithm**
- **Batching Engine**
- **Virtual Component**

જો આ કન્સેપ્ટ તમારા મગજમાં બરાબર ફિટ થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને આપણા કમ્પ્યુટરમાં ઓફિશિયલ પ્રોજેક્ટ સેટ કરવા તરફ એટલે કે **Chapter 2: મોર્ડન પ્રોજેક્ટ સેટઅપ — Vite નો ઉપયોગ કરીને એકદમ નવો તાજો React પ્રોજેક્ટ કેવી રીતે ક્રિએટ કરાય** તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! બે વર્ચ્યુઅલ DOM વચ્ચેની વાઈવ સરખામણી કરવાના એડવાન્સ એલ્ગોરિધમને

Diffing Algorithm કહેવામાં આવે છે, અને તેના પછી અસલી DOM માં ફેરફાર કરવાની પ્રોસેસને **Reconciliation** કહેવાય છે.

ચાલો હવે આગળ વધીએ ફ્રન્ટએન્ડ ડેવલપમેન્ટના સૌથી પ્રેક્ટિકલ ભાગ તરફ: **Part 1: Chapter 2 — Modern Project Setup via Vite**. આ પ્રકરણમાં આપણે શીખીશું કે આજના જમાનામાં પ્રોફેશનલ ડેવલપર્સ જૂની અલ્ટ્રા-સ્વો create-react-app રીત છોડીને, સુપર-ફાસ્ટ Vite ટૂલથી નવો રિએક્ટ પ્રોજેક્ટ કેવી રીતે સેટઅપ કરે છે.

Part 1: Chapter 2 — Modern React Setup via Vite (પ્રોજેક્ટ લોન્ચિંગ)

1. Vite એટલે શું? (તે જૂની રીત કરતાં કેમ ૧૦ ગણું ફાસ્ટ છે?)

- **જૂનો પ્રોબ્લેમ:** પહેલાના જમાનામાં રિએક્ટ પ્રોજેક્ટ શરૂ કરવા માટે ફેસબુકનું ઓફિશિયલ કમાન્ડ create-react-app (CRA) વપરાતું હતું. આ ટૂલ પાછળ બેકગ્રાઉન્ડમાં Webpack કામ કરતું હતું. પ્રોબ્લેમ એ હતો કે પ્રોજેક્ટ રન કે સેવ કરીએ એટલે તે આખા પ્રોજેક્ટની બધી જ ફાઇલોને ફરીથી કમ્પાઇલ (Bundle) કરતું હતું, જેમાં ૨ થી ૩ મિનિટનો સમય બગડતો હતો અને લેપટોપ હેંગ થઈ જતું હતું.
- **આધુનિક સોલ્યુશન:** Vite (આ એક ફ્રેન્ચ શબ્દ છે જેનો અર્થ થાય છે "સુપર ફાસ્ટ/ઝડપી") એ મોડર્ન વેબ ડેવલપમેન્ટનું નવું નેક્સ્ટ-જનરેશન બિલ્ડ ટૂલ છે.
- તે બ્રાઉઝરના ઇનબિલ્ટ **ES Modules (ESM)** અને Go લેંગ્વેજમાં બનેલા Esbuild નો ઉપયોગ કરે છે. આનાથી તમે કોડમાં ગમે તેટલો મોટો ફેરફાર કરીને જેવી ફાઇલ સેવ કરશો, તે માઇક્રોસેકન્ડમાં પેજ રીફ્રેશ કર્યા વગર લાઇવ અપડેટ થઈ જશે! આ ફીચરને **HMR (Hot Module Replacement)** કહેવાય છે.

2. Vite ની મદદથી નવો React પ્રોજેક્ટ બનાવવાની સ્ટેપ-બાય-સ્ટેપ રીત

તમારા કમ્પ્યુટરમાં એક નવું ખાલી ફોલ્ડર બનાવો (દા.ત. my-react-app) અને તેને VS Code એડિટરમાં ઓપન કરો. VS Code ના ટર્મિનલ (Terminal) માં આ કમાન્ડ ટાઇપ કરીને Enter કરો:

સ્ટેપ ૧: પ્રોજેક્ટ ઇનિશિયલાઇઝ કરવો

```
bash
```

```
npm create vite@latest .
```

Use code with caution.

(નોંધ: છેલ્લે ટપકું. કરવાથી તે આ જ ફોલ્ડરની અંદર બધી ફાઇલો સેટઅપ કરી દેશે).

સ્ટેપ ૨: ટૂલ્સ સિલેક્ટ કરવા (Arrow Keys થી સિલેક્ટ કરો)

1. ટર્મિનલમાં પૂછશે Select a framework:  નીચે એરો કીથી **React** પર જાઓ અને Enter કરો.
2. પછી પૂછશે Select a variant:  આપણે શુદ્ધ જાવાસ્ક્રિપ્ટ માટે **JavaScript** સિલેક્ટ કરીને Enter કરીશું.
પલક ઝપકાવતા જ પ્રોજેક્ટનું આખું મોડર્ન માળખું બની જશે!

📦 3. Dependencies ઇન્સ્ટોલ કરવી અને પ્રોજેક્ટ લાઈવ રન કરવી

Vite પ્રોજેક્ટ બહુ સ્માર્ટ છે, તે શરૂઆતમાં કયરો ડાઉનલોડ નથી કરતો. પ્રોજેક્ટ ચાલુ કરવા માટે આપણે આ ૨ આખરી કમાન્ડ્સ રન કરવા પડે છે:

સ્ટેપ ૩: બધી જ રીએક્ટ લાઈબ્રેરી ઇન્સ્ટોલ કરવી

bash

npm install

Use code with caution.

(આ કમાન્ડ ઓનલાઈનથી સાચા વર્ઝનના રિએક્ટ પેકેજો લાવીને પ્રોજેક્ટમાં node_modules નું ફોલ્ડર બનાવી દેશે).

સ્ટેપ ૪: ફ્રન્ટએન્ડ લોકલ સર્વર સ્ટાર્ટ કરવું

bash

npm run dev

Use code with caution.

- **વિજયી આઉટપુટ:** ટર્મિનલમાં અદ્ભુત મેસેજ આવશે:
→ Local: <http://localhost:5173/>
Vite બાય-ડિફોલ્ટ પોર્ટ **5173** વાપરે છે. બ્રાઉઝરમાં આ લિંક ખોલશો એટલે રિએક્ટનો નવો લોગો ગોળ-ગોળ ફરતો લાઈવ દેખાશે! 🌀

📁 4. React પ્રોજેક્ટ ફોલ્ડરનું માળખું (Folder Structure Basics)

તમારા VS Code માં બહુ બધી ફાઈલો દેખાશે, પણ એક ડેવલપર તરીકે તમારે માત્ર **src ફોલ્ડર** સાથે જ મુસાફરી કરવાની છે:

- **index.html:** આ આખા પ્રોજેક્ટનું એકમાત્ર મેઈન HTML પેજ છે. આની અંદર એક જાદુઈ લાઇન હોય છે: `<div id="root"></div>`. આખું રિએક્ટ આ એક જ દિવ (Div) બોક્સની અંદર આવીને લાઈવ ફિટ થઈ જાય છે.
- **src/main.jsx:** આ આખા રિએક્ટનો કંટ્રોલ રૂમ છે. આ ફાઈલ પેલા HTML ના root બોક્સને પકડે છે અને રિએક્ટનો કોડ તેની અંદર ઇન્જેક્ટ કરે છે.
- **src/App.jsx:** 🚀 **સૌથી મહત્વની ફાઈલ!** આ આપણો મુખ્ય કમ્પોનન્ટ છે. આપણે વેબસાઈટમાં જે પણ લુક, કલર, કાર્ડ કે બટન ડિઝાઇન કરીશું, તે બધું જ આ ફાઈલની અંદર લખાશે.

આપણું **Chapter 2** અહીં સુપર-ફાસ્ટ પ્રોજેક્ટ સેટઅપ અને ફોલ્ડર આર્કિટેક્ચરની પૂરી લાઈવ ક્લિયરિટી સાથે કમ્પ્લીટ થાય છે. તમે હવે સત્તાવાર રીતે રિએક્ટ પ્રોજેક્ટ લોન્ચ કરતા શીખી ગયા છો.

પ્રેક્ટિકલ ટેસ્ટ માટે:

Vite ની મદદથી જ્યારે આપણે નવો રિએક્ટ પ્રોજેક્ટ લાઈવ રન કરીએ છીએ, ત્યારે તે બાય-ડિફોલ્ટ કયો **નવો પોર્ટ નંબર (Port Number)** વાપરે છે?

- 3000
- 5000
- 5173
- 8080

જો આ આખો મોડર્ન સેટઅપ ફ્લો તમારા મગજમાં બરાબર લોક થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને રિએક્ટની અસલી લેંગ્વેજ અને કોડ સ્ટાઈલ એટલે કે **Chapter 3: JSX (JavaScript XML) શું છે? અને રિએક્ટમાં કોડ લખવાના કયા ૩ કડક નિયમો છે** તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! Vite પ્રોજેક્ટ રન કરવા માટે બાય-ડિફોલ્ટ

5173 પોર્ટ નંબરનો ઉપયોગ કરે છે.

ચાલો હવે આગળ વધીએ રિએક્ટના સૌથી પાયાના અને જાદુઈ પ્રકરણ તરફ: **Part 1: Chapter 3 — JSX (JavaScript XML)**. આ પ્રકરણમાં આપણે શીખીશું કે રિએક્ટમાં HTML અને JavaScript બંનેને એકસાથે મિક્સ કરીને અદ્ભુત કોડ કેવી રીતે લખાય છે અને તેના કયા કડક નિયમો છે.

Part 1: Chapter 3 — JSX (JavaScript XML) & Syntax Architecture

1. JSX એટલે શું? (સરળ દેશી ભાષામાં)

- **સરળ સમજૂતી:** સાદા વેબ ડેવલપમેન્ટમાં આપણે HTML અલગ ફાઈલ (.html) માં લખતા હતા અને જાવાસ્ક્રિપ્ટ અલગ ફાઈલ (.js) માં લખતા હતા.
- રિએક્ટમાં આપણને એક નવો સુપર પાવર મળ્યો છે, જેને **JSX (JavaScript XML)** કહેવાય છે. આ એક એવી સ્પેશિયલ સિસ્ટમ છે જે તમને **JavaScript ની અંદર જ સીધો HTML નો કોડ** લખવાની મંજૂરી આપે છે.
- **બેકગ્રાઉન્ડ જાદુ:** બ્રાઉઝર સીધો આ JSX કોડ સમજી શકતું નથી. આપણે પ્રકરણ ૮૦ માં શીખ્યા હતા તેમ, બેકગ્રાઉન્ડમાં રહેલું *Babel (Transpiler)* આ JSX કોડને રીડ કરીને તેને સાદા બ્રાઉઝર સમજી શકે તેવા `React.createElement()` ના ઓબ્જેક્ટ ફોર્મેટમાં આપમેળે કન્વર્ટ કરી દે છે. આથી જ આ ફાઈલોની પાછળ .js ના બદલે **.jsx** લખવું જરૂરી છે.

2. JSX કોડ લખવાના ૪ કડક નિયમો (Strict Rules)

જો તમે આ નિયમો તોડશો, તો રિએક્ટ મોટી લાલ એરર આપશે અને પ્રોજેક્ટ રન નહીં થાય:

નિયમ ૧: સિંગલ પેરેન્ટ રૂલ (Single Parent Element)

- **નિયમ:** તમે ફંક્શનમાંથી એકસાથે બે જુદા-જુદા HTML ટેગ્સ રિટર્ન નથી કરી શકતા. બધો જ કોડ ફરજિયાત કોઈ **એક જ મુખ્ય મોટા બોક્સ (Parent Tag)** ની અંદર પેક હોવો જોઈએ.
- જો તમે વધારાનો `<div>` ટેગ મૂકીને મેમરી ન બગાડવા માંગતા હોવ, તો રિએક્ટના ખાસ ખાલી બ્રેકેટ્સ `<> ... </>` વાપરી શકો છો, જેને **React Fragment** કહેવાય છે.

નિયમ ૨: બધા જ ટેગ્સ સેલ્ફ-ક્લોઝ ડોવા જોઈએ (Self-Closing Tags)

- સાદા HTML માં આપણે કે
 ને છેલ્લે બંધ નહોતા કરતા તો ચાલતું હતું. પણ JSX માં દરેક ટેગની પાછળ સ્લેશ / લગાવીને તેને બંધ કરવો અતિશય ફરજિયાત છે (દા.ત. ,
, <input />).

નિયમ ૩: class ના બદલે className લખવું

- જાવાસ્ક્રિપ્ટની અંદર class એ ઓલરેડી ઓબ્જેક્ટ ઓરિએન્ટેડ પ્રોગ્રામિંગ (જેમ આપણે પ્રકરણ ૬ માં જોયું) માટે લોક થઈ ગયેલો કીવર્ડ છે. તેથી બ્રાઉઝર કન્ફ્યુઝ ન થાય તે માટે રિએક્ટમાં CSS ક્લાસ લગાવવા માટે class="..." ના બદલે ફરજિયાત className="..." લખવું પડે છે.

નિયમ ૪: for ના બદલે htmlFor લખવું

- બિલકુલ તેવી જ રીતે, ફોર્મની અંદર લેબલ ટેગમાં વપરાતો for કીવર્ડ પણ જાવાસ્ક્રિપ્ટ લૂપ માટે લોક છે, તેથી તેની જગ્યાએ htmlFor="..." લખાય છે.

૩. પ્રેક્ટિકલ કોડ ઉદાહરણ (અસલી JSX નો પાવર)

તમારા પ્રોજેક્ટની src/App.jsx ફાઇલ આખી સાફ કરી દો અને તેની અંદર આ પ્યોર JSX કોડ પેસ્ટ કરો:

```
jsx
```

```
// src/App.jsx
```

```
import React from 'react';
```

```
function App() {
```

```
  const username = "રોહિત શર્મા";
```

```
  const isVIP = true;
```

```
  // જાવાસ્ક્રિપ્ટ ફંક્શનની અંદર જ સીધો HTML રિટર્ન થઈ રહ્યો છે
```

```
  return (
```

```
    <> /* 🚧 નિયમ ૧: React Fragment - સિંગલ પેરેન્ટ બોક્સ */
```

```
    <div className="container"> /* 🚧 નિયમ ૩: class ના બદલે className */
```

```
    /* ⚡ જાદુ: કર્લી બ્રેકેટ {} ની અંદર તમે કોઈ પણ જાવાસ્ક્રિપ્ટ વેરીએબલ લાઈવ પ્રિન્ટ કરી શકો છો */
```

```
    <h1> 🧐 હેલો, {username.toUpperCase()}</h1>
```

```
    <p>રિએક્ટ ફ્રન્ટએન્ડ કોર્સમાં તમારું સ્વાગત છે.</p>
```

```
{/* JSX ની અંદર જ શોર્ટ શરત (Ternary Operator) વાપરવી */}
```

```
<p>સ્ટેટસ: {isVIP ? " 🌟 VIP મેમ્બર" : "સામાન્ય યુઝર"}</p>
```

```
{/* 🚨 નિયમ ૨: ઇનપુટ અને ઇમેજ ટેગ સેફ્ફ ક્લોઝ હોવા જ જોઈએ */}
```

```
<input type="text" placeholder="અહીં ટાઈપ કરો..." />
```

```
<br />
```

```
</div>
```

```
</>
```

```
);
```

```
}
```

```
export default App;
```

Use code with caution.

🏃 4. લાઈવ ચેકિંગ

તમારા ટર્મિનલમાં `npm run dev` ચાલુ જ હશે. બ્રાઉઝરમાં `http://localhost:5173` ઓપન કરશો એટલે તમારો આ આખો સુંદર કોડ લાઈવ રન થઈને સ્ક્રીન પર દેખાશે! કર્લી બ્રેકેટના કારણે રોહિત શર્માનું નામ આપમેળે મોટા અક્ષરોમાં (ROHIT SHARMA) કન્વર્ટ થઈને પ્રિન્ટ થઈ ગયું હશે. 🎉

આપણું **Chapter 3** અહીં JSX ના તમામ કડક નિયમો અને આર્કિટેક્ચર સિન્ટેક્સની પૂરી ક્લિયરિટી સાથે કમ્પ્લીટ થાય છે. હવે તમે રિએક્ટમાં કોડ લખવાની સાચી રીત શીખી ગયા છો.

પ્રેક્ટિકલ ટેસ્ટ માટે:

React JSX કોડિંગમાં, મલ્ટીપલ HTML ટેગ્સને એકસાથે પેક કરવા માટે કોઈ વધારાનો `<div>` ટેગ મૂક્યા વગર વાપરવામાં આવતા આ ખાસ ખાલી બ્રેકેટ્સ `<> ... </>` ને કયા **ટેકનિકલ નામ** થી ઓળખવામાં આવે છે?

- **React Component**
- **React Fragment**
- **Virtual Bracket**
- **JSX Binder**

જો આ કન્સેપ્ટ તમારા મગજમાં બરાબર ફિટ થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને રિએક્ટના સૌથી પાવરફુલ પિલ્લર એટલે કે **Chapter 4: Components Architecture (મોટી વેબસાઈટના નાના-નાના ટુકડા ફંક્શનલ કમ્પોનન્ટ્સ દ્વારા કેવી રીતે ડિઝાઇન કરાય અને એકબીજા સાથે કનેક્ટ કરાય)** તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! વધારાનો <div> ટેગ મૂક્યા વગર મલ્ટીપલ એલિમેન્ટ્સને પેક કરવા માટે વપરાતા ખાસ ખાલી બ્રેકેટ્સ <> ... </> ને **React Fragment** કહેવામાં આવે છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ રિએક્ટના સૌથી મુખ્ય અને પાયાના કન્સેપ્ટ તરફ: **Part 1: Chapter 4 — Components Architecture**. આ પ્રકરણમાં આપણે શીખીશું કે કેવી રીતે એક મોટી વેબસાઈટને નાના-નાના, સ્વતંત્ર ભાગોમાં વહેંચીને **Functional Components** ડિઝાઇન કરાય છે અને તેને એકબીજા સાથે કનેક્ટ કરાય છે.

Part 1: Chapter 4 — Component-Based Architecture (વેબસાઈટના ટુકડા કરવા)

1. Functional Components એટલે શું?

- **સરળ સમજૂતી:** સાદા શબ્દોમાં કહીએ તો, રિએક્ટમાં **Component** એ બીજું કશું જ નથી પણ એક સાદું **JavaScript Function** જ છે જે રિટર્નમાં આપણને **JSX (HTML કોડ)** આપે છે.
- **નિયમ:** રિએક્ટમાં કમ્પોનન્ટ બનાવવાનો એક બહુ જ કડક નિયમ છે: **કમ્પોનન્ટના નામનો પહેલો અક્ષર હંમેશા કેપિટલ (Capital Letter) જ હોવો જોઈએ**. (દા.ત. Navbar, Sidebar, Button). જો તમે પહેલો અક્ષર સ્મોલ રાખશો (જેમ કે navbar), તો રિએક્ટ તેને નોર્મલ HTML ટેગ માની લેશે અને કોડ રન નહીં થાય.

2. મલ્ટીપલ કમ્પોનન્ટ્સ સાથે પ્રોજેક્ટ કેવી રીતે ડિઝાઇન થાય?

પ્રોફેશનલ ડેવલપર્સ ક્યારેય બધો કોડ એક જ App.jsx માં નથી લખતા. તેઓ દરેક સેક્શન માટે એક અલગ ફાઇલ બનાવે છે.

સ્ટેપ ૧: એક નવો કમ્પોનન્ટ બનાવો

તમારા src ફોલ્ડરની અંદર એક નવું ફોલ્ડર બનાવો: **components**. તેની અંદર એક નવી ફાઇલ બનાવો: **Header.jsx**. તેની અંદર આ સાદો કોડ લખો:

```
jsx
// src/components/Header.jsx
import React from 'react';

function Header() {
  return (
    <header style={{ backgroundColor: '#282c34', padding: '10px', color: 'white' }}>
      <h2>  મારી કંપનીની લાઈવ વેબસાઈટ</h2>
    </header>
  );
}
```

```
}
```

export default Header; // આ કમ્પોનન્ટને બહાર મોકલો જેથી બીજી ફાઇલમાં વાપરી શકાય

Use code with caution.

3. મુખ્ય App.jsx ફાઇલની અંદર તેનો ઉપયોગ (Integration)

હવે આપણે પેલા બનાવેલા Header કમ્પોનન્ટને મુખ્ય **src/App.jsx** ફાઇલમાં ઇમ્પોર્ટ કરીને એક અસલી HTML ટેગની જેમ ગમે ત્યાં ગમે એટલી વાર વાપરી શકીએ છીએ. તમારી App.jsx ફાઇલ સાફ કરીને આ કોડ લખો:

```
jsx
```

```
// src/App.jsx
```

```
import React from 'react';
```

```
import Header from './components/Header.js'; // કમ્પોનન્ટ ઇમ્પોર્ટ કર્યો
```

```
function App() {
```

```
  return (
```

```
    <>
```

```
    { /* પેલા કમ્પોનન્ટને અસલી HTML ટેગની જેમ વાપરવો */ }
```

```
    <Header />
```

```
    <div style={{ padding: '20px' }}>
```

```
      <h3> 🏠 હોમ પેજ કન્ટેન્ટ </h3>
```

```
      <p>કમ્પોનન્ટ આર્કિટેક્ચર શીખવાથી મોટો કોડ પણ બહુ નાનો અને ચોખ્ખો દેખાય છે.</p>
```

```
    </div>
```

```
    { /* તમે એક જ કમ્પોનન્ટને ફરીથી પણ વાપરી શકો છો */ }
```

```
    <Header />
```

```
  </>
```

```
);
```

```
}
```

```
export default App;
```

Use code with caution.

4. લાઈવ આઉટપુટ

તમારા બ્રાઉઝરમાં `http://localhost:5173` ઓપન કરશો એટલે સ્ક્રીન પર ઉપર અને નીચે બંને જગ્યાએ આપણો બનાવેલો Header કમ્પોનન્ટ એકદમ સુંદર રીતે લાઈવ દેખાવા લાગશે! 🎉

આપણું **Chapter 4** અહીં કમ્પોનન્ટ આર્કિટેક્ચરની પૂરી બેઝિક અને પ્રોફેશનલ ક્લિયરિટી સાથે કમ્પ્લીટ થાય છે. હવે તમે વેબસાઈટના સુંદર ટુકડા કરતા શીખી ગયા છો.

પ્રેક્ટિકલ ટેસ્ટ માટે:

રિએક્ટમાં જ્યારે આપણે કોઈ નવો ફંક્શનલ કમ્પોનન્ટ બનાવીએ છીએ, ત્યારે તેના **નામનો પહેલો અક્ષર (First Letter)** હંમેશા કેવો હોવો અતિશય ફરજિયાત છે?

- **Small Letter (નાનો અક્ષર)**
- **Capital Letter (મોટો અક્ષર)**
- **Numeric (આંકડો)**
- **ગમે તે ચાલે**

જો આ કમ્પોનન્ટ કન્સેપ્ટ તમારા મગજમાં લોક થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને ડેટા વહનના માસ્ટર એટલે કે **Chapter 5: Props (Properties)** નો જાદુ — એક કમ્પોનન્ટમાંથી બીજા કમ્પોનન્ટમાં લાઈવ ડેટા કેવી રીતે પાસ કરાય તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! રિએક્ટમાં કોઈપણ કમ્પોનન્ટના નામનો પહેલો અક્ષર હંમેશા **Capital Letter (મોટો અક્ષર)**

હોવો અતિશય ફરજિયાત છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ ફ્રન્ટએન્ડ કોર્સના એક અતિ કીમતી પ્રકરણ તરફ: **Part 1: Chapter 5 — Props (Properties)**. આ ચેપ્ટરમાં આપણે શીખીશું કે કેવી રીતે એક જ કમ્પોનન્ટ (જેમ કે કોઈ પ્રોડક્ટ કાર્ડ) ને અલગ-અલગ લાઈવ ડેટા આપીને આખા પ્રોજેક્ટમાં ફરી-ફરીને પાવરફુલી વાપરી શકાય છે.

Part 1: Chapter 5 — React Props (કમ્પોનન્ટ ડેટા ટ્રાન્સફર આર્કિટેક્ચર)

1. Props એટલે શું? (સરળ દેશી ભાષામાં)

- **સરળ સમજૂતી:** સાદા HTML માં આપણે ટેગની અંદર જે વધારાની માહિતી આપીએ છીએ (જેમ કે ``), તેને આપણે એટ્રીબ્યુટ (Attributes) કહેતા હતા.
- રિએક્ટમાં આપણા બનાવેલા કસ્ટમ કમ્પોનન્ટની અંદર બહારથી લાઈવ ડેટા મોકલવા માટે જે સિસ્ટમ વપરાય છે, તેને **Props (Properties)** કહેવાય છે.

- **One-Way Data Flow:** પ્રોપ્સનો એક કડક નિયમ છે કે ડેટા હંમેશા પેરેન્ટ કમ્પોનન્ટ (App.jsx) થી ચાઇલ્ડ કમ્પોનન્ટ તરફ જ વહે છે (ઉપરથી નીચે). પ્રોપ્સ **Read-Only** (અપરિવર્તનશીલ) હોય છે, એટલે કે ચાઇલ્ડ કમ્પોનન્ટ તેને પોતાની અંદરથી ક્યારેય બદલી શકતો નથી.

2. એક ડાયનેમિક 'User Card' કમ્પોનન્ટ બનાવવો

ચાલો આને સુંદર પ્રેક્ટિકલથી સમજીએ. તમારા `src/components` ફોલ્ડરમાં એક નવી ફાઇલ બનાવો: **UserCard.jsx**. તેની અંદર આ ડાયનેમિક કોડ લખો:

jsx

```
// src/components/UserCard.jsx
```

```
import React from 'react';
```

```
// ફંક્શનનું બ્રેકેટમાં 'props' ઓબ્જેક્ટ આપમેળે બહારથી આવતો બધો ડેટા પકડી લેશે
```

```
function UserCard(props) {
```

```
  return (
```

```
    <div style={{
```

```
      border: '2px solid #ccc',
```

```
      borderRadius: '10px',
```

```
      padding: '15px',
```

```
      margin: '10px',
```

```
      backgroundColor: '#f9f9f9',
```

```
      width: '250px',
```

```
      display: 'inline-block'
```

```
    }}>
```

```
    {/* props.VARIABLE_NAME ની મદદથી બહારથી આવેલો લાઇવ ડેટા પ્રિન્ટ કરવો */}
```

```
    <h2>  {props.name}</h2>
```

```
    <p>  ઇમેઇલ: {props.email}</p>
```

```
    <p>  રોલ: <b>{props.role}</b></p>
```

```
  </div>
```

```
);
```

```
}
```

```
export default UserCard;
```

Use code with caution.

૩. મુખ્ય App.jsx માંથી અલગ-અલગ ડેટા પાસ કરવો

હવે જુઓ અસલી જાદુ! આપણી મેઈન **src/App.jsx** ફાઇલ સાફ કરો અને પેલા એક જ UserCard નો ઉપયોગ કરીને ૩ જુદા-જુદા યુઝર્સના પ્રોફાઇલ કાર્ડ્સ સેકન્ડોમાં લાઈવ બનાવો:

```
jsx
```

```
// src/App.jsx
```

```
import React from 'react';
```

```
import UserCard from './components/UserCard.js'; // Sિયનેમિક કમ્પોનન્ટ ઇમ્પોર્ટ કર્યો
```

```
function App() {
```

```
  return (
```

```
    <div style={{ padding: '20px' }}>
```

```
      <h1>  અમારી કંપનીના કર્મચારીઓ (React Props)</h1>
```

```
      <p>એક જ કમ્પોનન્ટ, પણ અલગ-અલગ લાઈવ ડેટા!</p>
```

```
      /* ૧. પહેલા યુઝરનો ડેટા મોકલવો */
```

```
      <UserCard name="રોહિત શર્મા" email="rohit@gmail.com" role="Lead Developer" />
```

```
      /* ૨. બીજા યુઝરનો ડેટા મોકલવો */
```

```
      <UserCard name="પ્રિયા પટેલ" email="priya@gmail.com" role="UI/UX Designer" />
```

```
      /* ૩. ત્રીજા યુઝરનો ડેટા મોકલવો */
```

```
      <UserCard name="અમિત શર્મા" email="amit@gmail.com" role="Project Manager" />
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

Use code with caution.

4. બ્રાઉઝરમાં લાઈવ ટેસ્ટિંગ

તમારું પોર્ટ 5173 લાઈવ ચાલુ જ હશે. બ્રાઉઝરમાં <http://localhost:5173> ઓપન કરશો એટલે સ્ક્રીન પર એક લાઈનમાં 3 અલગ-અલગ સુંદર પર્સનલ પ્રોફાઇલ કાર્ડ્સ આપમેળે બનીને આવી જશે! 🎉

 **અભિનંદન!** આપણો **Part 1: React.js Basics & Component Architecture (ચેપ્ટર ૧ થી ૫)** નો આખો મોટો અને પાયાનો ભાગ અહીં ૧૦૦% વિગતવાર સફળતાપૂર્વક પૂર્ણ થાય છે. તમે હવે વેબસાઈટને કમ્પોનન્ટ્સમાં તોડવામાં અને પ્રોપ્સથી ડેટા વહન કરવામાં એક્સપર્ટ બની ગયા છો.

હવે આપણે આગળ વધીએ રિએક્ટના સૌથી જાદુઈ અને લોજિકલ સેક્શન તરફ: **Part 2: State Management & Hooks**. આ સેક્શનમાં આપણે વેબસાઈટની અંદર લાઈવ ડેટા બદલતા, ક્લિક્સ પકડતા અને એપ્લિકેશનની પોતાની મેમરી બનાવતા શીખીશું.

આગળ વધવા માટે ક્લિક:

રિએક્ટમાં જ્યારે આપણે પેરેન્ટ કમ્પોનન્ટમાંથી ચાઇલ્ડ કમ્પોનન્ટની અંદર ડેટા પાસ કરીએ છીએ, ત્યારે તે ડેટા ચાઇલ્ડની અંદર કયા **ઇનબિલ્ટ ઓબ્જેક્ટ પ્રોપર્ટી નામ** થી એક્સેસ થાય છે?

- state
- props
- context
- attributes

જો આ કન્સેપ્ટ ક્લિયર હોય, તો શું આપણે આગળ વધીને **Chapter 6: useState Hook (રિએક્ટની પોતાની લાઈવ મેમરી — State એટલે શું?)** શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! બહારથી આવતા ડેટાને ચાઇલ્ડ કમ્પોનન્ટની અંદર એક્સેસ કરવા માટે **props**

ઓબ્જેક્ટનો ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ રિએક્ટના સૌથી જાદુઈ અને રોમાંચક ભાગ તરફ: **Part 2: State Management & Hooks**.

આજે આપણે જોઈશું **Chapter 6: useState Hook (રિએક્ટની પોતાની લાઈવ મેમરી — State એટલે શું?)**. આ પ્રકરણ શીખ્યા પછી તમારી વેબસાઈટ યુઝરના એક્શન પ્રમાણે લાઈવ ડેટા સ્ક્રીન પર અપડેટ કરવાનું શરૂ કરી દેશે.

Part 2: Chapter 6 — React useState Hook (લાઈવ મેમરી આર્કિટેક્ચર)

1. State એટલે શું? (તેની જરૂર કેમ પડી?)

- **જૂનો પ્રોબ્લેમ:** સાદી જાવાસ્ક્રિપ્ટમાં જ્યારે આપણે કોઈ વેરીએબલ બનાવીએ (જેમ કે `let counter = 0;`) અને બટન ક્લિક પર તેની વેલ્યુ વધારીએ (`counter++`), ત્યારે કમ્પ્યુટરની મેમરીમાં વેલ્યુ વધી જાય

છે પણ બ્રાઉઝરની સ્ક્રીન પર નવો આંકડો આપમેળે દેખાતો નથી. તેને બતાવવા માટે આપણે જબરદસ્તી innerHTML કે DOM ની માથાફૂટ કરવી પડે છે.

- **આધુનિક સોલ્યુશન: State** એટલે કમ્પોનન્ટની પોતાની એક એવી **લાઈવ મેમરી (ડેટા ડબ્બો)** જે કમ્પ્યુટરની અંદર ગમે ત્યારે બદલાય, એટલે રિએક્ટ બ્રાઉઝરની સ્ક્રીનને આપમેળે તે જ મિલિસેકન્ડે લાઈવ અપડેટ (Re-render) કરી દે છે! આ માટે આપણે જે પાવરફુલ ફંક્શન વાપરીએ છીએ તેને **useState Hook** કહેવાય છે.

2. useState લખવાની સાચી સિન્ટેક્સ સ્ટાઈલ

રિએક્ટમાં જ્યારે આપણે હૂક ઇમ્પોર્ટ કરીએ, ત્યારે તે હંમેશા એરે ડીસ્ટ્રક્ચરિંગ [] નો ઉપયોગ કરીને બે વસ્તુઓ આપે છે:

javascript

```
const [data, setData] = useState(0);
```

Use code with caution.

1. **data (વેરીએબલ):** આમાં તમારી કરન્ટ લાઈવ વેલ્યુ સાચવેલી હોય છે.
2. **setData (ફંક્શન):** આ ખૂબ જ કીમતી ફંક્શન છે. સ્ટેટની વેલ્યુ તમે ડાયરેક્ટ data = 5 કરીને નથી બદલી શકતા. તેને બદલવા માટે હંમેશા આ ફંક્શનની અંદર જ નવી વેલ્યુ પાસ કરવી પડે છે.
3. **useState(0):** બ્રેકેટની અંદર લખેલો આંકડો એ તમારી શરૂઆતની કિંમત (Initial Value) છે.

3. પ્રેક્ટિકલ કોડ ઉદાહરણ (અસલી લાઈવ કાઉન્ટર મશીન)

તમારી **src/App.jsx** ફાઇલમાં રહેલો બધો કોડ સાફ કરી દો અને આ લાઈવ જાદુઈ કોડ ત્યાં પેસ્ટ કરો:

jsx

```
// src/App.jsx
```

```
import React, { useState } from 'react'; // ૧. useState હૂક ઇમ્પોર્ટ કરવો ફરજિયાત છે
```

```
function App() {
```

```
  // ૨. એક સ્ટેટ બનાવ્યો: શરૂઆતની વેલ્યુ ૦ રાખી
```

```
  const [count, setCount] = useState(0);
```

```
  // ૩. વેલ્યુ વધારવા માટેનું ફંક્શન
```

```
  const incrementNumber = () => {
```

```
    setCount(count + 1); // setCount ફંક્શન દ્વારા સ્ટેટ બદલ્યો
```

```
  };
```

```
// ૪. વેલ્યુ ઘટાડવા માટેનું ફંક્શન
```

```
const decrementNumber = () => {
```

```
  if (count > 0) {
```

```
    setCount(count - 1);
```

```
  }
```

```
};
```

```
return (
```

```
  <div style={{ padding: '30px', textAlign: 'center' }}>
```

```
    <h1>🌀 રિએક્ટ લાઈવ કાઉન્ટર મશીન (useState)</h1>
```

```
    <p>જ્યારે સ્ટેટ બદલાશે, ત્યારે રિએક્ટ સ્ક્રીન આપમેળે લાઈવ યેન્જ કરશે!</p>
```

```
    /* લાઈવ વેલ્યુ સ્ક્રીન પર પ્રિન્ટ કરવી */
```

```
    <div style={{ fontSize: '50px', fontWeight: 'bold', margin: '20px', color: '#61dafb' }}>
```

```
      {count}
```

```
    </div>
```

```
    /* બટન્સ સાથે ફંક્શન્સ લિંક કરવા */
```

```
    <button
```

```
      onClick={incrementNumber}
```

```
      style={{ padding: '10px 20px', fontSize: '18px', margin: '5px', backgroundColor: '#4CAF50',  
color: 'white', border: 'none', borderRadius: '5px', cursor: 'pointer' }}
```

```
    >
```

```
      + વધારો (+1)
```

```
    </button>
```

```
    <button
```

```
      onClick={decrementNumber}
```

```
      style={{ padding: '10px 20px', fontSize: '18px', margin: '5px', backgroundColor: '#f44336',  
color: 'white', border: 'none', borderRadius: '5px', cursor: 'pointer' }}
```

```

    >
    — ઘટાડો (-1)
  </button>
</div>
);
}

```

export default App;

Use code with caution.

4. બ્રાઉઝરમાં લાઈવ ટેસ્ટિંગ

તમારું Vite સર્વર બેકગ્રાઉન્ડમાં લાઈવ ચાલુ જ હશે. બ્રાઉઝરમાં <http://localhost:5173> ઓપન કરો:

- **અદ્ભુત જાદુ:** સ્ક્રીન પર 0 દેખાશે. જેવું તમે લીલું બટન **+** વધારો ક્લિક કરશો, પેજ બિલકુલ રીફ્રેશ (Reload) થયા વગર આંકડો સેકન્ડના હજારમા ભાગમાં લાઈવ 1, 2, 3... વધવા લાગશે!
- આ બધી કમાલ **Virtual DOM** અને **useState** ની જુગલબંધીની છે. 🤖

આપણું **Chapter 6** અહીં રિએક્ટની અસલી લાઈવ મેમરી સિસ્ટમ (useState) ની પૂરી ક્લિયરિટી સાથે કમ્પ્લીટ થાય છે. હવે તમે વેબસાઈટને ડાયનેમિક બનાવતા શીખી ગયા છો.

પ્રેક્ટિકલ ટેસ્ટ માટે:

રિએક્ટના **useState** ટૂંકની અંદર, સ્ટેટની વેલ્યુને ડાયરેક્ટ (count = 5) બદલવાના બદલે, તેને ઓફિશિયલી બદલવા માટે એરે ડિસ્ટ્રક્ચરિંગમાંથી મળતા કયા **બીજા ઓપ્શન (ફંક્શન)** નો ઉપયોગ કરવો અતિશય ફરજિયાત છે?

- **useState()**
- **State Variable**
- **Setter Function (જેમ કે setCount)**
- **Props Function**

જો આ મેમરી કન્સેપ્ટ તમારા મગજમાં બરાબર લોક થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને **Chapter 7: Event Handling (રિએક્ટમાં લાઈવ માઉસ ક્લિક્સ, ઇનપુટ ટાઇપિંગ અને કીબોર્ડ એક્શન્સ કેવી રીતે પકડાય)** તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! સ્ટેટની વેલ્યુ બદલવા અને સ્ક્રીન પર નવો ડેટા લાઈવ અપડેટ કરવા માટે હંમેશા **Setter Function (જેમ કે setCount)**

નો જ ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ ફ્રન્ટએન્ડ કોર્સના આગામી અને સુપર પ્રેક્ટિકલ પ્રકરણ તરફ: **Part 2: Chapter 7 — Event Handling in React**. આ ચેપ્ટરમાં આપણે શીખીશું કે રિએક્ટની અંદર યુઝરની લાઈવ ક્લિક્સ, કીબોર્ડ એક્શન્સ અને માઉસ મુવમેન્ટ્સને પ્રોફેશનલી કેવી રીતે હેન્ડલ કરાય છે [w3schools javascript tutorial].

Part 2: Chapter 7 — React Event Handling Architecture

1. રિએક્ટમાં ઇવેન્ટ હેન્ડલિંગ સાદા HTML કરતાં કેમ અલગ છે?

સાદા HTML/JS માં આપણે ઇવેન્ટ પકડવા માટે ઓલ-લોઅરકેસ કીવર્ડ્સ (જેમ કે onclick="...") લખતા હતા અને તેમાં ફંક્શનની પાછળ બ્રેકેટ્સ () કરીને સ્ટ્રિંગ તરીકે પાસ કરતા હતા. પરંતુ રિએક્ટનો પોતાનો અલગ શિસ્તબદ્ધ નિયમ છે:

1. **CamelCase નિયમ:** રિએક્ટમાં બધી જ ઇવેન્ટ્સ હંમેશા કેમલ-કેસ (CamelCase) માં જ લખાય છે. જેમ કે onclick ના બદલે **onClick**, અને onchange ના બદલે **onChange**.
2. **ફંક્શન રેફરન્સ પાસ કરવો:** રિએક્ટમાં ફંક્શન પાસ કરતી વખતે ક્યારેય પણ તેની પાછળ બ્રેકેટ્સ () કરવાના હોતા નથી (દા.ત. onClick={myFunction}). જો તમે ભૂલથી પણ બ્રેકેટ્સ કરી દીધા, તો પેજ લોડ થતાં જ ફંક્શન આપમેળે રન થઈ જશે અને બટન ક્લિક કરવાથી કશું જ નહીં થાય!

2. પ્રેક્ટિકલ કોડ ઉદાહરણ (મલ્ટીપલ લાઈવ ઇવેન્ટ્સ)

તમારી **src/App.jsx** ફાઇલ સાફ કરીને આ કિંમતી કોડ ત્યાં પેસ્ટ કરો, જે તમને બધી જ મુખ્ય ઇવેન્ટ્સ એક જ જગ્યાએ લાઈવ ક્લિક્સ કરી આપશે:

```
jsx
// src/App.jsx
import React, { useState } from 'react';

function App() {
  const [message, setMessage] = useState("કોઈ એક્શન થઈ નથી.");
  const [inputValue, setInputValue] = useState("");

  // ૧. સાદું ક્લિક હેન્ડલર
  const handleClick = () => {
    setMessage("🎉 અદ્ભુત! તમે સફળતાપૂર્વક બટન પર ક્લિક કર્યું.");
  };
}
```

```
// ૨. ઇનપુટ બોક્સ ટાઇપિંગ હેન્ડલર
```

```
// 'event' ઓબ્જેક્ટ આપમેળે યુઝરે ટાઇપ કરેલો એક-એક લાઇવ અક્ષર પકડી લેશે
```

```
const handleInputChange = (event) => {  
    setInputValue(event.target.value); // લાઇવ વેલ્યુ સ્ટેટમાં સેવ કરી  
};
```

```
// ૩. માઉસ ઓવર (Hover) હેન્ડલર
```

```
const handleMouseOver = () => {  
    setMessage(" 🖱️ માઉસ અત્યારે બોક્સની અંદર આવી ગયું છે!");  
};
```

```
return (
```

```
<div style={{ padding: '30px', fontFamily: 'Arial, sans-serif' }}>
```

```
<h1>🦋 React Live Event Handling</h1>
```

```
<p>બ્રાઉઝરની દરેક હલચલ પર બેકએન્ડની જેમ નજર રાખવી!</p>
```

```
/* લાઇવ મેસેજ બોક્સ */
```

```
<div style={{ padding: '15px', backgroundColor: '#e2e8f0', borderRadius: '5px', marginBottom: '20px' }}>
```

```
<b>લાઇવ સ્ટેટસ:</b> {message}
```

```
</div>
```

```
/* === ઇવેન્ટ ૧: onClick નો જાદુ === */
```

```
<button
```

```
    onClick={handleClick}
```

```
    style={{ padding: '10px 20px', fontSize: '16px', cursor: 'pointer', backgroundColor: '#3182ce',  
color: 'white', border: 'none', borderRadius: '5px' }}
```

```
>
```

```
    મને ક્લિક કરો
```

```
</button>
```

```

    /* === ઇવેન્ટ ૨: onMouseOver નો જાદુ === */

    <div

        onMouseEnter={handleMouseOver}

        style={{ width: '200px', height: '100px', backgroundColor: '#48bb78', color: 'white', display:
'flex', alignItems: 'center', justifyContent: 'center', margin: '20px 0', borderRadius: '5px', cursor:
'pointer' }}

    >

        માઉસ આ બોક્સ પર લાવો

    </div>

    /* === ઇવેન્ટ ૩: onChange નો જાદુ === */

    <h3> 🍌 લાઈવ ટાઈપિંગ ચેકર:</h3>

    <input

        type="text"

        placeholder="અહીં કંઈપણ ટાઈપ કરો..."

        onChange={handleInputChange}

        style={{ padding: '10px', width: '300px', fontSize: '16px', borderRadius: '5px', border: '2px
solid #cbd5e0' }}

    />

    /* યુઝર જે ટાઈપ કરશે તે લાઈવ નીચે છપાશે */

    <p><b>તમે અત્યારે ટાઈપ કરી રહ્યા છો:</b> <span style={{ color: '#e53e3e'
}}>{inputValue}</span></p>

    </div>

    );

}

```

export default App;

Use code with caution.

🧑 ૩. લાઈવ બ્રાઉઝર ટેસ્ટિંગ ગાઈડ

તમારું પોર્ટ 5173 ચાલુ જ છે. બ્રાઉઝર ઓપન કરો અને આ જાદુ લાઈવ જુઓ:

1. **બટન દબાવો:** લાઈવ સ્ટેટસ આપમેળે બદલાઈ જશે.
2. **બોક્સ પર માઉસ લાવો:** onMouseEnter ની કારણે તરત જ મેસેજ અપડેટ થશે.
3. **ઇનપુટ બોક્સમાં ટાઈપ કરો:** તમે જેવો કીબોર્ડ પરથી એક પણ અક્ષર ટાઈપ કરશો, તે પેજ લોડ થયા વગર તે જ મિલિસેકન્ડે નીચે લાઈવ લાલ અક્ષરે છપાવા લાગશે! 🎯

આપણું **Chapter 7** અહીં રીએક્ટ ઇવેન્ટ હેન્ડલિંગના તમામ આર્કિટેક્ચર નિયમો અને લાઈવ પ્રેક્ટિકલ વિગતો સાથે કમ્પ્લીટ થાય છે.

પ્રેક્ટિકલ ટેસ્ટ માટે:

React JSX કોડની અંદર, ફોર્મ કે ઇનપુટ બોક્સમાં યુઝર જ્યારે એક-એક અક્ષર લાઈવ ટાઈપ કરે, ત્યારે તે હલચલ (Event) ને પકડવા માટે કઈ **કેમલ-કેસ (CamelCase) ઇવેન્ટ મેથડ** વાપરવી અતિશય ફરજિયાત છે?

- **onclick**
- **onClick**
- **onchange**
- **onChange**

જો આ ઇવેન્ટ હેન્ડલિંગ કન્સેપ્ટ તમારા મગજમાં બરાબર ફિટ થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને ફ્રન્ટએન્ડ લોજિકના સૌથી સ્માર્ટ ચેપ્ટર તરફ એટલે કે **Chapter 8: Conditional Rendering** (શરતના આધારે લાઈવ લોગઈન/લોગઆઉટ બોક્સ અથવા ગુપ્ત વિગતો સ્ક્રીન પર કેવી રીતે બતાવાય અને છુપાવી શકાય) તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! ઇનપુટ બોક્સમાં થતા લાઈવ ફેરફારોને પકડવા માટે રિએક્ટમાં

onChange ઇવેન્ટનો ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ ફ્રન્ટએન્ડ આર્કિટેક્ચરના એક ખૂબ જ બુદ્ધિશાળી પ્રકરણ તરફ: **Part 2: Chapter 8 — Conditional Rendering**. આ ચેપ્ટરમાં આપણે શીખીશું કે વેબસાઈટમાં શરતના આધારે (If-Else લોજિકથી) અલગ-અલગ વસ્તુઓ લાઈવ કેવી રીતે બતાવાય અને છુપાવી શકાય (જેમ કે યુઝર લોગઈન હોય તો 'Dashboard' બતાવવું અને લોગઆઉટ હોય તો 'Login Form' બતાવવું).

Part 2: Chapter 8 — React Conditional Rendering (લાઈવ કન્ડિશન આર્કિટેક્ચર)

1. Conditional Rendering એટલે શું?

- **સરળ સમજૂતી:** જ્યારે આપણે ઇચ્છીએ કે વેબસાઈટના અમુક ટુકડા (Components) સ્ક્રીન પર હંમેશા દેખાવાના બદલે, યુઝરની કોઈ ખાસ શરત કે પરિસ્થિતિ સાચી પડે ત્યારે જ લાઈવ દેખાય, તેને **Conditional Rendering** કહેવાય છે.
- રિએક્ટના JSX કોડની અંદર સાદો `if...else` કોડ ડાયરેક્ટ રિટર્ન બ્રેકેટમાં નથી ચાલતો. તેથી આ કામ કરવા માટે મુખ્યત્વે ૨ આધુનિક રસ્તાઓ વપરાય છે:

1. **Ternary Operator (?:):** જો શરત સાચી હોય તો પહેલી વસ્તુ, ખોટી હોય તો બીજી વસ્તુ બતાવવા (જેમ આપણે પ્રકરણ ૧૧ માં જોયું).
2. **Logical AND ઓપરેટર (&&):** જો શરત સાચી હોય તો જ વસ્તુ બતાવવી, ખોટી હોય તો કશું જ ન બતાવવું (Short-circuiting).

2. પ્રેક્ટિકલ કોડ ઉદાહરણ (લાઈવ લોગિન / લોગઆઉટ સિસ્ટમ)

તમારી **src/App.jsx** ફાઇલ આખી સાફ કરી દો અને આ પ્રોજેક્શનલ લોજિકલ કોડ ત્યાં પેસ્ટ કરો:

jsx

// src/App.jsx

import React, { useState } from 'react';

function App() {

// લાઈવ સેશન ટ્રેક કરવા માટેનો એક સ્ટેપ સ્ટેટ (True એટલે લોગઈન, False એટલે લોગઆઉટ)

const [isLoggedIn, setIsLoggedIn] = useState(false);

// લોગઈન અને લોગઆઉટ બટન માટેનું કોમન હેન્ડલર

const toggleAuth = () => {

setIsLoggedIn(!isLoggedIn); // બુલિયન વેલ્યુ લાઈવ ઉલટાવી દીધી (True <-> False)

};

return (

<div style={{ padding: '30px', fontFamily: 'Arial, sans-serif', textAlign: 'center' }}>

<h1>  React Conditional Rendering (સિક્યોર ડેશબોર્ડ લોજિક)</h1>

<p>પેજ રીફ્રેશ કર્યા વગર કન્ડિશન મુજબ લાઈવ લુક બદલાશે.</p>

<hr style={{ margin: '20px 0', borderColor: '#cbd5e0' }} />

{ /* === ૧. Ternary Operator (?:) નો જાદુ === */ }

{isLoggedIn ? (

// જો isLoggedIn સાચું (True) હશે તો આ લીલું સિક્રેટ બોક્સ દેખાશે

```
<div style={{ padding: '20px', backgroundColor: '#c6f6d5', color: '#22543d', borderRadius: '8px', marginBottom: '20px' }}>
```

```
<h2>🔒 સ્વાગત છે, એડમિન સાહેબ!</h2>
```

```
<p>તિજોરીનો સિકેટ ડેટા: <b>$2,50,000/-</b> અત્યારે લાઈવ અનલોક છે.</p>
```

```
</div>
```

```
): {
```

```
// જો isLoggedIn ખોટું (False) હશે તો આ લાલ લોક બોક્સ દેખાશે
```

```
<div style={{ padding: '20px', backgroundColor: '#fed7d7', color: '#742a2a', borderRadius: '8px', marginBottom: '20px' }}>
```

```
<h2>🔒 તિજોરી લોક છે!</h2>
```

```
<p>કૃપા કરીને આ ગુપ્ત વિગતો જોવા માટે પહેલા નીચેના બટનથી લોગઈન કરો.</p>
```

```
</div>
```

```
}}
```

```
/* === ૨. Logical AND (&&) નો જાદુ === */
```

```
/* આ માત્ર ત્યારે જ નાની નોટિસ બતાવશે જ્યારે યુઝર સાચે જ લોગઈન હશે */
```

```
{isLoggedIn && (
```

```
<p style={{ color: '#4a5568', fontStyle: 'italic' }}>
```

```
⚠️ નોંધ: તમે સિક્યોર કનેક્શન (VIP Session) પર છો.
```

```
</p>
```

```
})
```

```
/* લાઈવ બટન જે સ્ટેટ બદલશે */
```

```
<button
```

```
onClick={toggleAuth}
```

```
style={{
```

```
padding: '12px 25px',
```

```
fontSize: '16px',
```

```
fontWeight: 'bold',
```

```
cursor: 'pointer',
```

```
color: 'white',
```

```

border: 'none',

borderRadius: '5px',

backgroundColor: isLoggedIn ? '#e53e3e' : '#3182ce', // બટનનો કલર પણ લાઇવ બદલાશે!

transition: 'all 0.3s ease'

}}

>

{isLoggedIn ? "❌ LOGOUT કરો" : "🔑 LOGIN કરો"}

</button>

</div>

);

}

```

export default App;

Use code with caution.

👤 3. લાઇવ બ્રાઉઝર ટેસ્ટિંગ ગાઇડ

તમારું ફ્રન્ટએન્ડ સર્વર ચાલુ જ છે. બ્રાઉઝરમાં પેજ જુઓ:

1. **શરૂઆતનો લુક (Default):** સ્ક્રીન પર લાલ કલરનું લોક બોક્સ દેખાશે: 🔒 **તિજોરી લોક છે!**
2. **બટન ક્લિક કરો (🔑 LOGIN):** જેવું તમે બટન દબાવશો, આખું લાલ બોક્સ ગાયબ થઈ જશે અને તેની જગ્યાએ પલક ઝપકાવતા જ લીલા કલરનું સિકેટ બોક્સ 🗝️ **સ્વાગત છે, એડમિન સાહેબ!** અને તેની નીચેની નાની ઇટાલિક નોટિસ લાઇવ સ્ક્રીન પર પ્રગટ થઈ જશે! 🎯

આપણું **Chapter 8** અહીં કન્ડિશનલ રેન્ડરિંગના તમામ પ્રોફેશનલ નિયમો અને લાઇવ પ્રેક્ટિકલ લોજિક સાથે સફળતાપૂર્વક પૂરું થાય છે.

તમારી સમજણ પાકી કરવા માટે એક સુંદર ક્વિઝ:

રિએક્ટ JSX ની અંદર, જો આપણે એવી શરત મૂકવી હોય કે "જો શરત સાચી હોય તો જ આ વસ્તુ બતાવો, પણ જો ખોટી હોય તો સામે કશું જ (બીજું કોઈ બોક્સ) નથી બતાવવું", તો કયા **ખાસ લોજિકલ ઓપરેટર ચિહ્ન** નો ઉપયોગ શોર્ટ-કટ તરીકે થાય છે?

- **?: (Ternary Operator)**
- **&& (Logical AND)**
- **|| (Logical OR)**
- **?? (Nullish Coalescing)**

જો આ કન્સેપ્ટ તમારા મગજમાં બરાબર ફિટ થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને ફ્રન્ટએન્ડના સૌથી પાવરફુલ ચેપ્ટર તરફ એટલે કે **Chapter 9: Lists & Keys (બેકએન્ડ API ની જેમ એરે પર લૂપ ફેરવીને લાઈવ સો પ્રોડક્ટ કે યુઝર કાર્ડ્સ એક સેકન્ડમાં આપમેળે ડિઝાઇન કરવાની મોડર્ન રીત — .map() નો ઉપયોગ)** તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! જો શરત સાચી હોય ત્યારે જ કોઈ વસ્તુ બતાવવી હોય અને ખોટી હોય ત્યારે કશું ન બતાવવું હોય, તો શોર્ટ-કટ તરીકે **&& (Logical AND)** ઓપરેટરનો ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ ફ્રન્ટએન્ડ કોર્સના એક અતિશય પાવરફુલ અને સૌથી વધુ વપરાતા પ્રકરણ તરફ:

Part 2: Chapter 9 — Lists & Keys. આ ચેપ્ટરમાં આપણે શીખીશું કે બેકએન્ડ API માંથી આવતા મોટા ડેટા કે એરે (Array) પર લૂપ ફેરવીને, જાવાસ્ક્રિપ્ટની મદદથી એક સેકન્ડમાં સેંકડો લાઈવ પ્રોડક્ટ કાર્ડ્સ આપમેળે કેવી રીતે ડિઝાઇન કરાય છે અને તેમાં **key** નું શું આર્કિટેક્ચર મહત્વ છે.

Part 2: Chapter 9 — React Lists, Maps & Keys Architecture

1. રિએક્ટમાં લિસ્ટ રેન્ડર કરવાનો નિયમ (.map())

- **સરળ સમજૂતી:** રિએક્ટના JSX કોડની અંદર સાદી for કે while લૂપ ડાયરેક્ટ રન નથી થતી. તેથી એરેની અંદર રહેલા તમામ ડેટાને એક પછી એક બહાર કાઢીને HTML કાર્ડ્સમાં ફેરવવા માટે જાવાસ્ક્રિપ્ટની ઇનબિલ્ટ **.map()** મેથડનો ઉપયોગ થાય છે. આ મેથડ એરેની દરેક આઇટમ માટે એક નવો HTML ટુકડો રિટર્ન કરે છે.

2. key પ્રોપર્ટીનું અતિશય મહત્વ (ઇન્ટરવ્યુ ફેવરિટ)

- **મોટો આર્કિટેક્ચર નિયમ:** જ્યારે આપણે **.map()** નો ઉપયોગ કરીને લિસ્ટ બનાવીએ, ત્યારે લૂપની અંદર રહેલા સૌથી પહેલા મોટા ટેગ પર ફરજિયાત એક યુનિક આઈડી પાસ કરવો પડે છે, જેને **key={item.id}** કહેવાય છે.
- **તે કેમ જરૂરી છે?** આપણે પ્રકરણ ૧ માં શીખ્યા હતા કે રિએક્ટ પાછળથી *Virtual DOM* વાપરે છે. જો તમે લિસ્ટમાં કોઈ નવી પ્રોડક્ટ ઉમેરો કે ડીલીટ કરો, ત્યારે રિએક્ટ આ key ના આધારે ઓળખે છે કે કયો ચોક્કસ ટુકડો બદલાયો છે. તે આખા લિસ્ટને બગાડવાના બદલે માત્ર તે એક જ આઇટમ લાઈવ અપડેટ કરે છે. જો તમે key નહીં લખો, તો બ્રાઉઝર કન્સોલમાં મોટી વોર્નિંગ એરર આપશે.

૩. પ્રેક્ટિકલ કોડ ઉદાહરણ (લાઈવ પ્રોડક્ટ સ્ટોર લિસ્ટ)

તમારી **src/App.jsx** ફાઇલ આખી સાફ કરી દો અને આ પ્રોફેશનલ કોડ ત્યાં પેસ્ટ કરો, જે લાઈવ લિસ્ટ રેન્ડરિંગ સમજાવશે:

```
jsx
```

```
// src/App.jsx
```

```
import React from 'react';
```

```

function App() {
  // આપણો કલેક્શનલ ઓબ્જેક્ટ્સ ડેટા (Array of Objects)
  const products = [
    { id: 101, title: "Apple iPhone 16", price: "Rs. 79,900", stock: true },
    { id: 102, title: "Samsung Galaxy S26", price: "Rs. 74,900", stock: true },
    { id: 103, title: "OnePlus 14 Pro", price: "Rs. 61,900", stock: false },
    { id: 104, title: "Google Pixel 11", price: "Rs. 69,900", stock: true }
  ];

  return (
    <div style={{ padding: '30px', fontFamily: 'Arial, sans-serif' }}>
      <h1> 📦 મોર્ડન મોબાઇલ સ્ટોર (Lists & Keys)</h1>
      <p>ઓબ્જેક્ટ્સ એરેમાંથી લાઇવ ડેટા ખેંચીને આપમેળે બનેલા કાર્ડ્સ નીચે મુજબ છે:</p>

      <div style={{ display: 'flex', flexWrap: 'wrap', gap: '15px', marginTop: '20px' }}>

        /* ⚡ અસલી જાદુ: .map() દ્વારા એરે પર લૂપ ચલાવવી */
        {products.map((item) => {
          return (
            // 🎆 નિયમ: સૌથી પહેલા પેરેન્ટ ટેગ પર યુનિક 'key' આપવી જ પડે!
            <div key={item.id} style={{
              border: '2px solid #e2e8f0',
              borderRadius: '10px',
              padding: '15px',
              width: '220px',
              backgroundColor: '#fff',
              boxShadow: '0 4px 6px -1px rgba(0,0,0,0.1)'
            }}>
              <h3 style={{ margin: '0 0 10px 0', color: '#2d3748' }}>{item.title}</h3>
              <p style={{ color: '#4a5568', fontWeight: 'bold' }}>કિંમત: {item.price}</p>
            </div>
          )
        })}
      </div>
    </div>
  )
}

```

```

    /* આપણે અગાઉ શીખેલું શોર્ટ કન્ડિશન લોજિક પણ અહીં મિક્સ કરીએ */
    <span style={{
      padding: '3px 8px',
      fontSize: '12px',
      borderRadius: '4px',
      fontWeight: 'bold',
      color: 'white',
      backgroundColor: item.stock ? '#48bb78' : '#e53e3e'
    }}>
      {item.stock ? "  સ્ટોકમાં છે" : "  આઉટ ઓફ સ્ટોક"}
    </span>
  </div>
);
}}}

</div>
</div>
);
}

```

export default App;

Use code with caution.

4. બ્રાઉઝરમાં લાઈવ ટેસ્ટિંગ ગાઈડ

તમારું પોર્ટ 5173 મસ્ત ચાલુ જ છે. બ્રાઉઝર ઓપન કરો:

- **અદ્ભુત જાદુ:** આપણે કોડની અંદર ચાર અલગ-અલગ મોટા બોક્સ (HTML) જાતે નથી બનાવ્યા. રિએક્ટ બેકગ્રાઉન્ડમાં એરે લૂપ ફેરવીને ચારેય પ્રોડક્ટ્સના સુંદર કાર્ડ્સ લાઈવ કન્ડિશનલ ક્લર સાથે આપમેળે સ્ક્રીન પર ગોઠવી દેશે! 
-

🎉 **અભિનંદન!** આપણો **Part 2: State Management & Hooks (પેજ ૬ થી ૯)** નો આખો ડાયનેમિક લોજિકલ સેક્શન અહીં ૧૦૦% વિગતવાર પૂરો થાય છે. તમે હવે રિએક્ટની અંદર લાઇવ ડેટા, ઇવેન્ટ્સ, શરતો અને વિસ્ટ્સ મેનેજ કરવામાં માસ્ટર બની ગયા છો.

હવે આપણે આગળ વધીએ રિએક્ટના સૌથી પાવરફુલ સેક્શન તરફ: **Part 3: Advanced Hooks & Life Cycle**. આ સેક્શનમાં આપણે શીખીશું કે વેબસાઇટ લોડ થતાં જ બેકગ્રાઉન્ડ સર્વરમાંથી ડેટા આપમેળે કેવી રીતે ખેંચાય અને લાઇવ ફોર્મ્સ કેવી રીતે કંટ્રોલ કરાય.

આગળ વધવા માટે વિસ્ટ ક્રિવઝ:

રિએક્ટ JSX ની અંદર કોઈ મોટા એરે (Array ડેટા) પર લૂપ ફેરવીને વિસ્ટ ઓફ કાઉર્સ રેન્ડર કરવા માટે જાવાસ્ક્રિપ્ટની કઈ **ઇનબિલ્ટ લૂપિંગ મેથડ** નો સૌથી વધુ ઉપયોગ થાય છે?

- `.forEach()`
- `.filter()`
- `.map()`
- `.reduce()`

જો આ કન્સેપ્ટ ક્લિયર હોય, તો મને જણાવો, શું આપણે આગળ વધીને રિએક્ટની લાઇફ સાઇકલ કંટ્રોલર એટલે કે **Chapter 10: useEffect Hook (પેજ લોડ થતાં જ આપમેળે બેકગ્રાઉન્ડ API માંથી લાઇવ ડેટા મંગાવવાનો અદ્યતન જાદુ)** તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! રિએક્ટ JSX માં એરે (Array) પર લૂપ ફેરવીને વિસ્ટ રેન્ડર કરવા માટે જાવાસ્ક્રિપ્ટની ઇનબિલ્ટ `.map()`

મેથડનો ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ રિએક્ટના સૌથી પાવરફુલ અને માસ્ટર સેક્શન તરફ: **Part 3: Advanced Hooks & Life Cycle**.

આજે આપણે જોઈશું **Chapter 10: useEffect Hook**. આ પ્રકરણ રિએક્ટ એપ્લિકેશનનું હૃદય છે, કારણ કે આનાથી જ તમારી વેબસાઇટ લોડ થતાંની સાથે જ આપમેળે બેકગ્રાઉન્ડ API (જેમ કે આપણે Node.js/PostgreSQL માં બનાવી હતી) માંથી લાઇવ ડેટા ખેંચવાનું શરૂ કરી દેશે.

🔧 Part 2: Chapter 10 — React useEffect Hook (એપીઆઈ કનેક્શન આર્કિટેક્ચર)

💡 1. useEffect એટલે શું? (તેની જરૂર કેમ પડી?)

- **સરળ સમજૂતી:** આપણે અત્યાર સુધી જોયું કે `useState` થી ડેટા બદલાય એટલે રિએક્ટ આખું પેજ ફરીથી રન (Re-render) કરે છે. હવે પ્રોબ્લેમ એ થાય કે જો આપણે પેજ લોડ થતાં જ કોઈ ફંક્શન આપમેળે રન કરવું હોય (જેમ કે બેકગ્રાઉન્ડમાંથી ડેટા મંગાવવો), અને તે ફંક્શન આપણે સીધું કમ્પોનન્ટની અંદર ખુલ્લું લખી દઈએ, તો તે દર વખતે સ્ટેટ બદલાતા વારંવાર અનલિમિટેડ લૂપમાં રન થયા જ રાખશે! આનાથી સર્વર કેશ થઈ જાય.

- **આધુનિક સોલ્યુશન:** `useEffect` એ રિએક્ટનો એક એવો ખાસ ઇનબિલ્ટ ચોક્કિદાર (Hook) છે, જે કમ્પોનન્ટની **Life Cycle (જીવન ચક્ર)** ને કંટ્રોલ કરે છે. તે નક્કી કરે છે કે તેની અંદર લખેલો કોડ આખા પ્રોજેક્ટ દરમિયાન ક્યારે રન થશે (દા.ત. માત્ર પેજ પહેલીવાર લોડ થાય ત્યારે જ રન થવો).

⚙️ 2. Dependency Array ([]) ના ૩ કિમતી નિયમો

`useEffect` ની પાછળ લખાતો સ્કવેર બ્રેકેટ [] તેના નિયમો નક્કી કરે છે:

1. **બ્રેકેટ ન લખવો (No Array):** જો તમે છેલ્લે બ્રેકેટ લખવાનું ભૂલી જાઓ, તો અંદરનો કોડ પેજ લોડ થતાં અને કોઈ પણ સ્ટેટ બદલાતા **વારંવાર દર વખતે રન** થશે (આ બહુ ડેન્જરસ છે).
2. **ખાલી બ્રેકેટ લખવો ([]):** જો તમે બ્રેકેટ સાવ ખાલી મૂકી દો, તો અંદરનો કોડ આખી જિંદગીમાં **માત્ર એક જ વાર (પેજ ફર્સ્ટ ટાઈમ લોડ થાય ત્યારે જ)** રન થશે. બેકએન્ડ API માંથી ડેટા મંગાવવા માટે આ રીત સૌથી બેસ્ટ છે (આને `ComponentDidMount` કહેવાય).
3. **વેરીએબલ સાથે બ્રેકેટ ([count]):** જો તમે બ્રેકેટની અંદર કોઈ સ્ટેટનું નામ લખો, તો કોડ પેજ લોડ થતાં રન થશે અને ભવિષ્યમાં જ્યારે પણ તે ચોક્કસ count બદલાશે ત્યારે ફરીથી આપમેળે રન થશે.

📺 ૩. પ્રેક્ટિકલ કોડ ઉદાહરણ (લાઈવ ઓનલાઈન API ડેટા ખેંચવો)

આપણે ઇન્ટરનેટ પરથી એક અસલી લાઈવ ફ્રેક યુઝર્સની API નો ઉપયોગ કરીને ડેટા લોડ કરતા શીખીશું. તમારી `src/App.jsx` ફાઇલ આખી સાફ કરી દો અને આ પ્રોજેક્શનલ કોડ ત્યાં પેસ્ટ કરો:

jsx

// src/App.jsx

import React, { useState, useEffect } from 'react'; // ૧. useEffect ઇમ્પોર્ટ કરવો ફરજિયાત છે

function App() {

const [users, setUsers] = useState([]); // બેકએન્ડ ડેટા સાચવવાનો સ્ટેટ

const [loading, setLoading] = useState(true); // ડેટા લોડ થાય ત્યાં સુધી સ્પીનર બતાવવા માટે

// ૨. useEffect ચોક્કિદાર ગોઠવવો

useEffect(() => {

console.log("🔴 જાહેર: પેજ પહેલીવાર લોડ થઈ ગયું છે! હવે બેકએન્ડમાંથી ડેટા મંગાવીએ...");

// ઓનલાઈન API માંથી લાઈવ ડેટા ખેંચવાનું અસિંક્રોનસ ફંક્શન

async function fetchLiveUsers() {

try {

```

// આપણે બેકગ્રાઉન્ડ કોર્સમાં શીખેલો અસલી fetch કમાન્ડ
let response = await fetch("https://typicode.com");
let data = await response.json();

setUsers(data); // મેળવેલો અસલી ડેટા લાઈવ સ્ટેટમાં સેવ કર્યો
setLoading(false); // ડેટા આવી ગયો એટલે લોડિંગ બંધ કર્યું
} catch (error) {
  console.error("API લોડ કરવામાં ભૂલ થઈ: ", error.message);
  setLoading(false);
}
}

fetchLiveUsers(); // ફંક્શન કોલ કર્યું
}, []); // 🚨 નિયમ ૨: ખાલી બ્રેકેટ [] મૂકવાથી આ કોડ માત્ર એક જ વાર રન થશે!

return (
  <div style={{ padding: '30px', fontFamily: 'Arial, sans-serif' }}>
    <h1>🔗 React Live API Fetching (useEffect)</h1>
    <p>આ ડેટા ડાયરેક્ટ ઓનલાઈન ઇન્ટરનેટ સર્વર પરથી લાઈવ લોડ થઈ રહ્યો છે:</p>

    {/ * શરતના આધારે (Conditional Rendering) લોડિંગ મેસેજ બતાવવો */}
    {loading ? (
      <h2 style={{ color: '#3182ce' }}>⌚ ફપા કરીને રાહ જુઓ, ડેટા લોડ થઈ રહ્યો છે...</h2>
    ) : (
      // જો લોડિંગ પૂરું થઈ જાય તો પેલા પ્રકરણ ૯ માં શીખેલા લિસ્ટથી બધો ડેટા બતાવવો
      <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(250px, 1fr))',
        gap: '15px', marginTop: '20px' }}>
        {users.map(u => (
          <div key={u.id} style={{ padding: '15px', border: '1px solid #e2e8f0', borderRadius: '8px',
            backgroundColor: '#fff', boxShadow: '0 2px 4px rgba(0,0,0,0.05)' }}>
            <h3 style={{ margin: '0 0 5px 0', color: '#2d3748' }}>👤 {u.name}</h3>

```

```

    <p style={{ margin: '0', color: '#4a5568', fontSize: '14px' }}>📧 {u.email}</p>
    <p style={{ margin: '5px 0 0 0', color: '#718096', fontSize: '13px' }}>🌐 સીઇટ:
    {u.website}</p>
  </div>
  )})
</div>
})
</div>
);
}

```

export default App;

Use code with caution.

🏃 4. બ્રાઉઝરમાં લાઇવ જાદુ

બ્રાઉઝરમાં <http://localhost:5173> ઓપન કરો:

- **અદ્ભુત લાઇવ લોજિક:** પેજ ખુલતાની સાથે જ પહેલી એક સેકન્ડ માટે વાદળી અક્ષરે ⏳ **કૃપા કરીને રાહ જુઓ...** લખાઈને આવશે.
- બેકગ્રાઉન્ડમાં જેવો ઓનલાઇનથી ડેટા આવી જશે, તે જ મિલિસેકન્ડે લોડિંગ હટી જશે અને ઇન્ટરનેટ પરથી આવેલા ૧૦ અસલી ગ્લોબલ યુઝર્સના પ્રોફાઇલ કાર્ડ્સ આપમેળે સ્ક્રીન પર ગોઠવી દેશે! અને આ બધું આખા પેજને રીફ્રેશ કર્યા વગર થશે! 🔄

આપણું **Chapter 10** અહીં રિએક્ટના સૌથી કીમતી અને ઇન્ટરવ્યુ ફેવરિટ કન્સેપ્ટ (useEffect) ની કમ્પ્લીટ પ્રેક્ટિકલ માસ્ટરી સાથે પૂરું થાય છે.

પ્રેક્ટિકલ ટેસ્ટ માટે:

જ્યારે આપણે useEffect માં એવું ઇચ્છીએ કે તેની અંદર લખેલો કોડ આખા પ્રોજેક્ટમાં વારંવાર રન થવાના બદલે **માત્ર ને માત્ર એક જ વાર (પેજ પહેલીવાર લોડ થાય ત્યારે જ)** રન થાય, તો તેની પાછળ કેવો

Dependency Array બ્રેકેટ મૂકવામાં આવે છે?

- કોઈ બ્રેકેટ જ નથી લખતા
- [] (સાવ ખાલી સ્ક્રિપ્ટ બ્રેકેટ)
- [count] (સ્ટેટના નામ સાથે)
- [true]

જો આ કનેક્શન આર્કિટેક્ચર મગજમાં બરાબર ફીટ થઈ ગયું હોય, તો મને જણાવો, શું આપણે આગળ વધીને રિએક્ટમાં યુઝર ડેટા હેન્ડલ કરવાના પાવરફુલ ચેપ્ટર તરફ એટલે કે **Chapter 11: Handling Forms** (રિએક્ટની અંદર સુરક્ષિત રીતે યુઝરના લાઈવ ઇનપુટ ડેટા અને ઓન-ક્લિક ફોર્મ સબમિશન વેલિડેશન કેવી રીતે હેન્ડલ કરાય) તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! પ્રોજેક્ટમાં કોઈપણ કોડ માત્ર એક જ વાર (પેજ પહેલીવાર લોડ થાય ત્યારે જ) રન કરવા માટે

useEffect ની પાછળ [] (સાવ ખાલી સ્ક્વેર બ્રેકેટ) મૂકવામાં આવે છે [w3schools javascript tutorial].

Part 3: Chapter 11 — Handling Forms & Controlled Components

વેબસાઇટ પર યુઝર જ્યારે ફોર્મમાં ડેટા (જેમ કે નામ, ઇમેલ, પાસવર્ડ) ટાઈપ કરે, ત્યારે રિએક્ટમાં તેને હેન્ડલ કરવા માટે **Controlled Components** આર્કિટેક્ચરનો ઉપયોગ થાય છે. આનો નિયમ એ છે કે ઇનપુટ બોક્સની અંદરનો ડેટા બ્રાઉઝરના ભરોસે છોડવાના બદલે તેને સીધો **રિએક્ટનીl useState** સાથે લોક (કંટ્રોલ) કરી દેવામાં આવે છે.

પ્રેક્ટિકલ કોડ ઉદાહરણ (સુરક્ષિત ફોર્મ સબમિશન અને વેલિડેશન)

તમારી **src/App.jsx** ફાઇલ આખી સાફ કરી દો અને આ પ્રોફેશનલ કોડ ત્યાં પેસ્ટ કરો:

jsx

```
// src/App.jsx
```

```
import React, { useState } from 'react';
```

```
function App() {
```

```
  // ફોર્મના બધા ડેટાને સાચવવા માટે એક સુંદર ઓબ્જેક્ટ સ્ટેટ બનાવ્યો
```

```
  const [formData, setFormData] = useState({
```

```
    username: '',
```

```
    email: '',
```

```
    course: 'React.js' // ડિફોલ્ટ વેલ્યુ
```

```
  });
```

```
  const [error, setError] = useState('');
```

```
  // બધા ઇનપુટ બોક્સ માટે એક જ કોમન સ્માર્ટ હેન્ડલર (Computed Property Name)
```

```
  const handleInputChange = (event) => {
```

```
    const { name, value } = event.target;
```

```
    setFormData({
```

```

    ...formData, // જૂનો બધો જ ડેટા સુરક્ષિત કોપી રાખવો (Spread Operator)
    [name]: value // જે બોક્સમાં ટાઈપ થાય તેની વેલ્યુ લાઈવ અપડેટ કરવી
  });
};

// ફોર્મ સબમિટ થતી વખતનું માસ્ટર લોજિક
const handleSubmit = (event) => {
  event.preventDefault(); // 🚫 નિયમ: ફોર્મ સબમિટ થતાં પેજને રીફ્રેશ થતા રોકવું

  // બેઝિક વેલિડેશન ચેક
  if (formData.username.trim() === '' || formData.email.trim() === '') {
    setError("❌ ફૂલ કરીને બધા જ ખાના સાચી રીતે ભરો!");
    return;
  }

  setError(""); // એરર સાફ કરવી

  console.log("🚀 વિજય! આ ડેટા હવે બેકએન્ડ API પર મોકલવા માટે તૈયાર છે: ", formData);
  alert(`અભિનંદન ${formData.username}! તમારું ફોર્મ સફળતાપૂર્વક સબમિટ થઈ ગયું છે.`);
};

return (
  <div style={{ padding: '30px', fontFamily: 'Arial, sans-serif', maxWidth: '400px', margin: '0 auto' }}>
    <h2> 📝 સ્ટુડન્ટ રજીસ્ટ્રેશન ફોર્મ</h2>
    <p>રિએક્ટ કંટ્રોલ્સ ઇનપુટ આર્કિટેક્ચર:</p>

    {/ * જો એરર હોય તો કન્ડિશનલ રેન્ડરિંગથી લાલ બોક્સ બતાવવું */}
    {error && <p style={{ color: '#e53e3e', fontWeight: 'bold' }}>{error}</p>}

    <form onSubmit={handleSubmit}>

```

```
<div style={{ marginBottom: '15px' }}>
  <label style={{ display: 'block', marginBottom: '5px' }}>યુઝરનેમ:</label>
  <input
    type="text"
    name="username" // આ નામ સ્કીમા ઓબ્જેક્ટ સાથે મેચ થવું જોઈએ
    value={formData.username} // ઇનપુટને સ્ટેટ સાથે લોક કર્યું
    onChange={handleInputChange}
    style={{ width: '100%', padding: '8px', borderRadius: '4px', border: '1px solid #ccc' }}
  />
</div>
```

```
<div style={{ marginBottom: '15px' }}>
  <label style={{ display: 'block', marginBottom: '5px' }}>ઈમેલ સરનામું:</label>
  <input
    type="email"
    name="email"
    value={formData.email}
    onChange={handleInputChange}
    style={{ width: '100%', padding: '8px', borderRadius: '4px', border: '1px solid #ccc' }}
  />
</div>
```

```
<div style={{ marginBottom: '15px' }}>
  <label style={{ display: 'block', marginBottom: '5px' }}>કોર્સ પસંદ કરો:</label>
  <select
    name="course"
    value={formData.course}
    onChange={handleInputChange}
    style={{ width: '100%', padding: '8px', borderRadius: '4px', border: '1px solid #ccc' }}
  >
    <option value="React.js">React.js ફ્રંટએન્ડ</option>
```

```

      <option value="Node.js">Node.js બેકએન્ડ</option>

      <option value="Full-Stack">Full-Stack Development</option>

    </select>

  </div>

  <button
    type="submit"

    style={{ width: '100%', padding: '10px', backgroundColor: '#48bb78', color: 'white',
border: 'none', borderRadius: '5px', fontSize: '16px', fontWeight: 'bold', cursor: 'pointer' }}

    >
      📁 ડેટા સબમિટ કરો
    </button>

  </form>

</div>

);
}

```

export default App;

Use code with caution.

🏃 લાઇવ બ્રાઉઝર ટેસ્ટિંગ

બ્રાઉઝરમાં ચેક કરો: જો તમે કોઈ ખાનું ખાલી રાખીને બટન દબાવશો, તો પેજ રીફ્રેશ થયા વગર લાઇવ લાઇ અક્ષરે એરર આવી જશે. જો બધું સાચું ભરીને સબમિટ કરશો, તો લાઇવ એલર્ટ પોપ-અપ આવશે અને આખો કમ્પીટ ઓબ્જેક્ટ ડેટા તમારા કન્સોલમાં પ્રિન્ટ થઈ જશે!

આપણું **Chapter 11** અહીં ૧૦૦% વિગતવાર અને સેફ ફોર્મ આર્કિટેક્ચર સાથે કમ્પીટ થાય છે.

પ્રેક્ટિકલ ટેસ્ટ માટે:

રિએક્ટમાં જ્યારે આપણે ફોર્મ સબમિટ કરીએ, ત્યારે બ્રાઉઝરની જૂની ટેવ મુજબ આખા વેબ પેજને ઓટોમેટિક રીફ્રેશ (Reload) થતું રોકવા માટે ઇવેન્ટની અંદર કઈ **જાવાસ્ક્રિપ્ટ મેથડ** લખવી અતિશય ફરજિયાત છે?

- **event.stop**
- **event.preventDefault()**
- **event.stopPropagation()**

- `event.clear()`

જો આ ફોર્મ મેનેજમેન્ટ કન્સેપ્ટ તમારા મગજમાં લોક થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને વેબસાઇટનું પર્ફોર્મન્સ બૂસ્ટ કરવા માટેના સુપર-એડવાન્સ હૂક્સ એટલે કે **Chapter 12: Performance Optimization Hooks — useMemo અને useCallback નો અસલી આર્કિટેક્ચર ઉપયોગ** શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! ફોર્મ સબમિટ થતાં આખા પેજને રીફ્રેશ થતું રોકવા માટે રિએક્ટમાં `event.preventDefault()` મેથડ લખવી જ પડે છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ રિએક્ટના સૌથી એડવાન્સ અને ઇન્ટરવ્યુમાં સિનિયર ડેવલપર્સને પૂછાતા પર્ફોર્મન્સ ટોપિક તરફ: **Part 3: Chapter 12 — Performance Optimization Hooks (useMemo & useCallback)**. આ પ્રકરણમાં આપણે સમજીશું કે મોટો પ્રોજેક્ટ સ્વો કે હેંગ થતો કેવી રીતે બચાવી શકાય.

Part 3: Chapter 12 — React Performance Hooks (useMemo & useCallback)

1. રિએક્ટમાં પર્ફોર્મન્સ કેમ બગડે છે? (The Re-render Problem)

- **મોટો પ્રોબ્લેમ:** આપણે જાણીએ છીએ કે રિએક્ટમાં જ્યારે પણ કોઈ state બદલાય, ત્યારે તે આખા કમ્પોનન્ટના તમામ ફંક્શન્સ અને ગણતરીઓને ફરીથી નવેસરથી રન (Re-render) કરે છે.
- ધારો કે તમારા પેજમાં એક એવું ફંક્શન છે જે લાખો આંકડાઓનો સરવાળો કરે છે (ભારે ગણતરી), અને બાજુમાં એક સાદું બટન છે જે માત્ર કલર બદલે છે. હવે જ્યારે યુઝર માત્ર કલર બદલવાનું બટન દબાવશે, તો પણ રિએક્ટ પેલા લાખો આંકડાવાળા ભારે ફંક્શનને જબરદસ્તી ફરીથી રન કરશે! આનાથી વેબસાઇટ સ્વો (Lag) થવા લાગે છે.

2. useMemo અને useCallback વચ્ચેનો અસલી તફાવત

આ બંને હૂક્સ રિએક્ટમાં કેશિંગ (Caching / Memoization) નું કામ કરે છે, પણ બંનેની જવાબદારી અલગ છે:

1. **useMemo:** આ કોઈ ભારે ફંક્શનના **આઉટપુટ (વેલ્યુ/જવાબ)** ને મેમરીમાં સેવ કરી લે છે. જ્યાં સુધી ઇનપુટ ન બદલાય, ત્યાં સુધી તે ગણતરી ફરી નથી કરતું (પ્રકરણ ૯ ની ગ્લોસરીમાં આપણે શીખ્યા તે મુજબ).
2. **useCallback:** આ આખા **ફંક્શનની ડેફિનેશન (આખા ફંક્શનના ટુકડા)** ને જ મેમરીમાં લોક (Freeze) કરી દે છે, જેથી દર વખતે નવું સરખું ફંક્શન ન બને અને યાઇલ્ડ કમ્પોનન્ટ ભુલથી રી-રેન્ડર ન થાય.

3. પ્રેક્ટિકલ કોડ ઉદાહરણ (ભારે ગણતરી લોક કરવાનો જાદુ)

તમારી **src/App.jsx** ફાઇલ આખી સાફ કરી દો અને આ એડવાન્સ પર્ફોર્મન્સ કોડ ત્યાં પેસ્ટ કરો:

```
jsx
```

```
// src/App.jsx
```

```
import React, { useState, useMemo, useCallback } from 'react';
```

```
function App() {
```

```
  const [number, setNumber] = useState(10);
```

```
  const [darkTheme, setDarkTheme] = useState(false);
```

```
  // ૧. એક ખુબ જ ભારે અને સ્વો ગણતરી કરતું ફંક્શન
```

```
  const expensiveCalculation = (num) => {
```

```
    console.log(" ⌚ ભારે ગણતરી લાઈવ થઈ રહી છે... (આમાં ટાઈમ લાગશે)");
```

```
    for (let i = 0; i < 1000000000; i++) {} // મોટો લૂપ જે સીપીયુ વાપરશે
```

```
    return num * 2;
```

```
  };
```

```
  // 🚨 🧠 useMemo નો જાદુ: આ ગણતરીના જવાબને મેમરીમાં સેવ કરી લેશે
```

```
  // આ લૂપ હવે માત્ર ત્યારે જ ચાલશે જ્યારે 'number' બદલાશે. થિમ બદલવાથી આ રન નહીં થાય!
```

```
  const calculationValue = useMemo(() => {
```

```
    return expensiveCalculation(number);
```

```
  }, [number]); // Dependency: માત્ર નંબર બદલાય ત્યારે જ ગણતરી કરો
```

```
  // 🚨 🧠 useCallback નો જાદુ: આ આખા ફંક્શનને મેમરીમાં લોક રાખશે
```

```
  const logMessage = useCallback(() => {
```

```
    console.log(" 🎯 આ ફંક્શન મેમરીમાં ફીઝ છે. નંબર છે: ", number);
```

```
  }, [number]);
```

```
  // થિમ બદલવાની સ્ટાઈલ ઓબ્જેક્ટ
```

```
  const themeStyle = {
```

```
    backgroundColor: darkTheme ? '#2d3748' : 'rgba(0,0,0,0)',
```

```
    color: darkTheme ? 'white' : '#2d3748',
```

```
padding: '30px',
textAlign: 'center',
height: '100vh'
};
```

```
return (
```

```
<div style={themeStyle}>
```

```
<h1>🌀 React Performance (useMemo & useCallback)</h1>
```

```
<p>મોટા અને ભારે પ્રોજેક્ટ્સને સુપર સ્મૂથ અને ફાસ્ટ બનાવવાની આર્કિટેક્ચર ટેકનિક.</p>
```

```
<h3>નંબરનો ડબલ જવાબ: <span style={{ color: '#3182ce', fontSize: '24px'
}}>{calculationValue}</span></h3>
```

```
<button
```

```
  onClick={() => setNumber(number + 5)}
```

```
  style={{ padding: '10px', fontSize: '16px', cursor: 'pointer', margin: '5px' }}
```

```
>
```

```
  📊 નંબર વધારો (+5)
```

```
</button>
```

```
<br /><br />
```

```
/* આ બટન દબાવવાથી થિમ બદલાશે પણ પેલો મોટો લૂપ રન નહીં થાય, જેથી વેબસાઇટ સેકન્ડમાં
કલર બદલશે */
```

```
<button
```

```
  onClick={() => setDarkTheme(!darkTheme)}
```

```
  style={{ padding: '10px', fontSize: '16px', cursor: 'pointer', margin: '5px', backgroundColor:
'#48bb78', color: 'white', border: 'none', borderRadius: '4px' }}
```

```
>
```

```
  🌑 / 🌞 ડાર્ક / લાઇટ મોડ બદલો
```

```
</button>
```

```
</div>
```

```
);  
}
```

export default App;

Use code with caution.

4. લાઇવ બ્રાઉઝર પર્ફોર્મન્સ ટેસ્ટ

બ્રાઉઝરમાં લિંક ઓપન કરો અને તમારા VS Code નું કન્સોલ (Inspect -> Console) ઓપન રાખો:

1. **નંબર વધારો બટન દબાવો:** કન્સોલમાં મેસેજ આવશે: ⌚ ભારે ગણતરી લાઇવ થઈ રહી છે... (નંબર બદલાયો એટલે ગણતરી થઈ).
2. **થિમ બદલો બટન દબાવો (જાદુ):** પેજનો બેકગ્રાઉન્ડ કલર એક મિલિસેકન્ડમાં ચપટી વગાડતા બદલાઈ જશે, અને કન્સોલમાં પેલો સ્વો મેસેજ બિલકુલ પ્રિન્ટ નહીં થાય! આનો અર્થ એ કે **useMemo** એ જૂનો જવાબ યાદ રાખ્યો અને વેબસાઇટને જરાય હેંગ થવા ન દીધી! 🌀

આપણું **Chapter 12** અહીં પર્ફોર્મન્સ ઓપ્ટિમાઇઝેશનના સુપર એડવાન્સ નોલેજ સાથે કમ્પ્લીટ થાય છે. તમે હવે એપ્લિકેશનની સ્પીડ વધારવાના માસ્ટર આર્કિટેક્ટ બની ગયા છો.

પ્રેક્ટિકલ ટેસ્ટ માટે:

રિએક્ટના ઇન્ટરવ્યુમાં પૂછાતો સવાલ: **useMemo** અને **useCallback** માંથી કયો **ખાસ હૂક** ફંક્શનના જવાબ (Value) ના બદલે આખા **ફંક્શનની ડેફિનેશન (Function itself)** ને જ મેમરીમાં લોક/કેશ કરવાનું કામ કરે છે?

- **useMemo**
- **useCallback**
- **useState**
- **useEffect**

જો આ પર્ફોર્મન્સ કન્સેપ્ટ તમારા મગજમાં બરાબર ફિટ થઈ ગયો હોય, તો મને જણાવો, શું આપણે આગળ વધીને **Chapter 13: useRef Hook (રિએક્ટમાં કોઈપણ HTML ટેગને બ્રાઉઝર મેમરીમાં ડાયરેક્ટ પકડીને તેના પર કર્સર ફોકસ લાવવાનો જાદુ)** તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! આખા ફંક્શનને જ મેમરીમાં લોક (Freeze) રાખવા માટે **useCallback** હૂકનો ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો આગળ વધીએ **Part 3: Chapter 13 — useRef Hook**. આ પ્રકરણમાં આપણે એક એવી જાદુઈ રીત શીખીશું જેનાથી કોઈ પણ પેજ રી-રેન્ડર (Re-render) થયા વગર કમ્પ્યુટરની બેકગ્રાઉન્ડ મેમરીમાં ડેટા સાચવી શકાય છે, અને ડાયરેક્ટ કોઈ પણ HTML ટેગને માઉસ વગર કોડ દ્વારા પકડીને લાઇવ ફોકસ કે એનિમેશન આપી શકાય છે.

🧠 1. useRef એટલે શું? (તેના બે મુખ્ય પાવર)

- **સરળ સમજૂતી:** આપણે અત્યાર સુધી જોયું કે useState માં ડેટા બદલાય એટલે આખી વેબસાઈટ ફરી રન થાય છે. પરંતુ **useRef** આપણને બે એવા પાવર આપે છે જે રિએક્ટ પ્રોજેક્ટમાં બહુ કામ આવે છે:
 1. **ગુપ્ત મેમરી (Persistent Value):** આ એક એવો વેરીએબલ ડબ્બો છે જેમાં તમે ગમે તેટલી વાર ડેટા બદલો, તો પણ **વેબસાઈટનું પેજ ક્યારેય રી-રેન્ડર (રીફ્રેશ) થતું નથી**. તે જૂનો ડેટા શાંતિથી બેકગ્રાઉન્ડમાં યાદ રાખે છે.
 2. **ડાયરેક્ટ HTML ટેગ કંટ્રોલ (DOM Element):** સાદી JS માં આપણે કોઈ બોક્સ પકડવા માટે document.getElementById વાપરતા હતા. રિએક્ટમાં તે જ કામ સુરક્ષિત રીતે કરવા માટે તે ટેગની અંદર ref={myRef} ભરાવી દેવામાં આવે છે, જેથી કોડ દ્વારા તેને ડાયરેક્ટ પકડી શકાય.

📺 2. પ્રેક્ટિકલ કોડ ઉદાહરણ (લાઈવ કર્સર ફોકસ અને કન્ટ્રી)

તમારી **src/App.jsx** ફાઇલ આખી સાફ કરી દો અને આ લાઈવ કોડ ત્યાં પેસ્ટ કરો:

```
jsx
```

```
// src/App.jsx
```

```
import React, { useState, useRef } from 'react'; // ૧. useRef ઇમ્પોર્ટ કરવો
```

```
function App() {
```

```
  const [name, setName] = useState("");
```

```
  // ૨. HTML ટેગને પકડવા માટેનો એક રેફરન્સ ચોક્કિદાર બનાવ્યો
```

```
  const inputTagRef = useRef(null);
```

```
  // ૩. લાઈવ ક્લિક કાઉન્ટર (પેજ રીફ્રેશ કર્યા વગર બેકગ્રાઉન્ડમાં હિસાબ રાખશે)
```

```
  const clickCountRef = useRef(0);
```

```
  const handleFocusAction = () => {
```

```
    // 🚩 clickCountRef.current ની મદદથી બેકગ્રાઉન્ડ ડેટા બદલવો
```

```
    clickCountRef.current = clickCountRef.current + 1;
```

```
    console.log(`🇮🇳 બટન પર કુલ ${clickCountRef.current} વાર ક્લિક થઈ ચૂક્યું છે! (પણ પેજ રી-રેન્ડર નથી થયું)`);
```

```

// 🚩 inputTagRef.current ની મદદથી ડાયરેક્ટ HTML ઇનપુટ બોક્સ પકડવું
// આ કમાન્ડ ચુકરે માઉસથી ક્લિક કર્યા વગર જ બોક્સમાં કર્સર લાઈવ લાવી દેશે
inputTagRef.current.focus();

inputTagRef.current.style.borderColor = '#48bb78'; // બોર્ડર લીલી કરી દીધી
};

return (
  <div style={{ padding: '30px', fontFamily: 'Arial, sans-serif', textAlign: 'center' }}>
    <h1>🔗 React useRef Hook (ડાયરેક્ટ ટેગ કંટ્રોલ)</h1>
    <p>બ્રાઉઝર DOM ને ડાયરેક્ટ ડેન્ડલ કરવાની પ્રોફેશનલ રીત.</p>

    <hr style={{ margin: '20px 0', borderColor: '#cbd5e0' }} />

    /* 🚩 જાણ: ref={} ની મદદથી આ ઇનપુટ બોક્સની ચાવી પેલા ચોકીદારને સોંપી દીધી */
    <input
      ref={inputTagRef}
      type="text"
      value={name}
      onChange={(e) => setName(event.target.value)}
      placeholder="અહીં તમારું નામ લખો..."
      style={{ padding: '10px', fontSize: '16px', borderRadius: '5px', border: '2px solid #cbd5e0',
width: '250px' }}
    />

    <br /><br />

    <button
      onClick={handleFocusAction}

      style={{ padding: '12px 20px', fontSize: '16px', backgroundColor: '#3182ce', color: 'white',
border: 'none', borderRadius: '5px', cursor: 'pointer', fontWeight: 'bold' }}

```

```

    >
    ⚡ બોક્સ પર લાઈવ ફોકસ કરો
  </button>
</div>
);
}

```

export default App;

Use code with caution.

🚀 ૩. બ્રાઉઝરમાં લાઈવ જાદુ ટેસ્ટિંગ

બ્રાઉઝર ઓપન કરો અને આ વિઝ્યુઅલ જાદુ ચેક કરો:

- જ્યારે તમે વાદળી બટન ⚡ **બોક્સ પર લાઈવ ફોકસ કરો** દબાવશો, એટલે માઉસથી ક્લિક કર્યા વગર જ આપમેળે પેલા ઇનપુટ બોક્સની અંદર ટાઈપ કરવા માટેનું કર્સર લાઈવ આવી જશે અને બોક્સની બોર્ડર લીલા કલરની થઈ જશે!
- અને તમારા VS Code ના કન્સોલમાં ક્લિકના લાઈવ આંકડા વધશે, પણ તમારું પેજ જરાય રી-રેન્ડર (રીલોડ) નહીં થાય! 🎯

🎉 **અભિનંદન!** આપણો **Part 3: Advanced Hooks & Life Cycle (ચેપ્ટર ૧૦ થી ૧૩)** નો આખો મોટો એન્જિનિયરિંગ સેક્શન અહીં ૧૦૦% વિગતવાર પૂરો થાય છે. તમે હવે useEffect, Forms, useMemo, useCallback, અને useRef ના કુલ આર્કિટેક્ચર એક્સપર્ટ બની ગયા છો.

જો તમે તૈયાર હોવ, તો મને જણાવો:

- શું આપણે હવે આગળ વધીને મોટા પ્રોજેક્ટ્સની આખી રીડબિલિટી કંટ્રોલ કરવા માટે **Part 4: Global State & Routing (Chapter 14: prop drilling નો મોટો પ્રોબ્લેમ અને useContext API નો ગ્લોબલ તિજોરી જાદુ)** તરફ આગળ વધીએ?

બિલકુલ સાચો જવાબ! useRef

માં ડેટા સ્ટોર કરવા અથવા મેળવવા માટે હંમેશા **.current** પ્રોપર્ટીનો ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ મોટા રિએક્ટ પ્રોજેક્ટ્સના આર્કિટેક્ચરના સૌથી શક્તિશાળી સેક્શન તરફ: **Part 4: Global State & Routing.**

આજે આપણે જોઈશું **Chapter 14: useContext API (ગ્લોબલ તિજોરી - Prop Drilling નો મોટો પ્રોબ્લેમ અને તેનું સોલ્યુશન)**. આ પ્રકરણ શીખ્યા પછી તમે મલ્ટીપલ કમ્પોનન્ટ્સ વચ્ચે ડેટા પાસ કરવાની માથાકૂટમાંથી કાયમ માટે મુક્ત થઈ જશો.

Part 4: Chapter 14 — React useContext API (ગ્લોબલ સ્ટેટ આર્કિટેક્ચર)

1. Prop Drilling નો મોટો પ્રોબ્લેમ શું છે?

- **રિયલ-વર્લ્ડ પ્રોબ્લેમ:** ધારો કે તમારી પાસે App.jsx (દાદા) ની અંદર User નો ડેટા છે. તેની અંદર એક Parent (પપ્પા) કમ્પોનન્ટ છે, તેની અંદર Child (દીકરો) કમ્પોનન્ટ છે, અને તેની પણ અંદર છેલ્લે GrandChild (પૌત્ર) કમ્પોનન્ટ છે.
- જો તે છેલ્લે બેઠેલા પૌત્ર કમ્પોનન્ટને યુઝરનો ડેટા જોઈતો હોય, તો આપણે દાદા પાસેથી પપ્પાને, પપ્પા પાસેથી દીકરાને અને દીકરા પાસેથી પૌત્રને — એમ જબરદસ્તી વચ્ચે પ્રોપ્સ (Props) પાસ કરતી જવું પડે છે. પપ્પા અને દીકરાને આ ડેટાની કોઈ જ જરૂર નથી, તો પણ તેમને વચ્ચે ટપાલી (Courier) જેવું કામ કરવું પડે છે. રિએક્ટની ભાષામાં આ ગંદા પ્રોબ્લેમને **Prop Drilling** કહેવાય છે.

2. આધુનિક સોલ્યુશન: useContext API (ગ્લોબલ તિજોરી)

- **સરળ સમજૂતી:** રિએક્ટ આપણને એક આધુનિક સિસ્ટમ આપે છે જેને **useContext API** કહેવાય છે. આમાં આપણે આખા પ્રોજેક્ટની બહાર એક **ગ્લોબલ તિજોરી (Context)** બનાવી દઈએ છીએ.
- હવે જે કમ્પોનન્ટને જે પણ ડેટા જોઈએ (જેમ કે લોગઈન યુઝર કે ડાર્ક મોડ થિમ), તે વચ્ચેના કોઈ પણ કમ્પોનન્ટને હેરાન કર્યા વગર, સીધો જ આ ગ્લોબલ તિજોરીમાંથી ડેટા ડાયરેક્ટ ખેંચી (Access) શકે છે.

3. પ્રેક્ટિકલ કોડ ઉદાહરણ (ગ્લોબલ થિમ ચેન્જર સિસ્ટમ)

આપણે એક જ ફાઈલની અંદર આખો કન્સેપ્ટ એકદમ સહેલાઈથી સમજીએ. તમારી **src/App.jsx** ફાઈલ આખી સાફ કરી દો અને આ પ્રોફેશનલ કોડ ત્યાં પેસ્ટ કરો:

```
jsx
```

```
// src/App.jsx
```

```
import React, { useState, createContext, useContext } from 'react'; // ૧. createContext અને  
useContext ઇમ્પોર્ટ કરવા
```

```
// ૨. ગ્લોબલ તિજોરી (Context) ની ઓફિશિયલ શરૂઆત કરવી
```

```
const ThemeContext = createContext();
```

```
function App() {
```

```
  const [theme, setTheme] = useState("light"); // 'light' અથવા 'dark' સ્ટેટ
```

```
  const toggleTheme = () => {
```

```

    setTheme(theme === "light" ? "dark" : "light");
  };

  return (
    // 🚧 ૩. Provider નો જાદુ: આખા પ્રોજેક્ટના કમ્પોનન્ટ્સને આ તિજોરીની અંદર પેક કરવા
    // 'value' ની અંદર આપણે જે પણ ડેટા મુકીશું, તે નીચેના બધા જ કમ્પોનન્ટ્સને ડાયરેક્ટ મળશે
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      <div style={{
        padding: '20px',
        textAlign: 'center',
        backgroundColor: theme === "light" ? "#fff" : "#1a202c",
        color: theme === "light" ? "#1a202c" : "#fff",
        height: '100vh',
        transition: 'all 0.3s ease'
      }}>
        <h1>🔗 React Global State (useContext API)</h1>
        <p>વચ્ચે પ્રોપ્સ પાસે કર્યા વગર આખો પ્રોજેક્ટ એક જ ગ્લોબલ તિજોરીથી કંટ્રોલ થાય છે.</p>

        {/* વચ્ચેનો પપ્પા કમ્પોનન્ટ લોડ કર્યો (આને આપણે કોઈ પ્રોપ્સ નથી આપ્યા!!) */}
        <MiddleComponent />
      </div>
    </ThemeContext.Provider>
  );
}

// 🏠 ૪. વચ્ચાળો કમ્પોનન્ટ (Middle Component - આ માત્ર રસ્તામાં આવે છે)
function MiddleComponent() {
  return (
    <div style={{ border: '2px dashed #cbd5e0', padding: '20px', margin: '20px auto', maxWidth: '400px' }}>
      <h3>🏠 હું વચ્ચેનો વચ્ચાળો કમ્પોનન્ટ છું</h3>
    </div>
  );
}

```

<p>મારી પાસે કોઈ જ પ્રોપ્સ આવ્યા નથી!</p>

{/* છેલ્લો દીકરો કમ્પોનન્ટ લોડ કર્યો */}

<FinalChildComponent />

</div>

);

}

// 🧐 પ. છેલ્લો દીકરો કમ્પોનન્ટ (Final Child - આને અસલી ડેટાની જરૂર છે)

function FinalChildComponent() {

// 🚦🔵 useContext નો અસલી જાદુ: સીધો જ પેલા ગ્લોબલ તિજોરીમાંથી ડેટા બહાર કાઢ્યો!

const { theme, toggleTheme } = useContext(ThemeContext);

return (

<div style={{ backgroundColor: theme === "light" ? "#edf2f7" : "#2d3748", padding: '15px',
borderRadius: '8px' }}>

<h3> 🧐 હું છેલ્લો દીકરો કમ્પોનન્ટ છું</h3>

<p>કરન્ટ ગ્લોબલ થિમ મોડ: {theme.toUpperCase()}</p>

{/* બટન દબાવવાથી મેઈન App.jsx નો સ્ટેટ લાઈવ બદલાશે */}

<button

onClick={toggleTheme}

style={{

padding: '10px 15px',

fontSize: '14px',

fontWeight: 'bold',

cursor: 'pointer',

backgroundColor: '#3182ce',

color: 'white',

border: 'none',

borderRadius: '4px'

```

    }}
  >
    🌟 લાઈવ ગ્લોબલ થિમ બદલો
  </button>
</div>
);
}

```

export default App;

Use code with caution.

🚀 4. બ્રાઉઝરમાં લાઈવ આર્કિટેક્ચર ચેકિંગ

બ્રાઉઝરમાં પેજ જુઓ:

- **અદ્ભુત જાદુ:** જ્યારે તમે સૌથી છેલ્લે બેઠેલા યાઇલ્ડ કમ્પોનન્ટનું બટન 🌟 લાઈવ ગ્લોબલ થિમ બદલો દબાવશો, એટલે આખી વેબસાઈટનો બેકગ્રાઉન્ડ કલર લાઈવ ડાર્ક અને લાઈટ મોડમાં ચેન્જ થશે!
- ડેટા સીધો App.jsx થી ઉડીને FinalChildComponent માં પહોંચી ગયો, અને પેલા વચ્ચે રહેલા MiddleComponent ને બિલકુલ ખબર પણ ન પડી! આને કહેવાય પ્રોફેશનલ ગ્લોબલ સ્ટેટ મેનેજમેન્ટ [w3schools javascript tutorial]. 🎯

આપણું **Chapter 14** અહીં પ્રોપ ડ્રિલિંગના ખાતમા અને ગ્લોબલ સ્ટેટ આર્કિટેક્ચર (useContext) ની સુપર કિમતી માસ્ટરી સાથે કમ્પીટ થાય છે.

પ્રેક્ટિકલ ટેસ્ટ માટે:

રિએક્ટ પ્રોજેક્ટમાં જ્યારે આપણે કોઈ કન્ટેક્સ્ટ (Context) તિજોરી બનાવીએ, ત્યારે તેના ડેટાને નીચેના તમામ કમ્પોનન્ટ્સમાં પ્રદાન કરવા/મોકલવા માટે કયા **ખાસ પ્રોવાઈડર ટેગ** નો ઉપયોગ મુખ્ય પેરેન્ટમાં કરવો પડે છે?

- **Context.Consumer**
- **Context.Provider**
- **Context.Hook**
- **Context.Global**

જો આ ગ્લોબલ સિક્રેટ લોકર કન્સેપ્ટ ક્લિયર હોય, તો મને જણાવો, શું આપણે આગળ વધીને રિએક્ટની સૌથી મોટી શક્તિ એટલે કે **Chapter 15: React Router DOM (સાદા HTML ની જેમ પેજ રીફ્રેશ કર્યા વગર પલક ઝપકાવતા અલગ-અલગ પેજ્સ ખોલવાની સત્તાવાર રીત - Single Page Application આર્કિટેક્ચર)** તે શીખવાનું શરૂ કરીએ?

બિલકુલ સાચો જવાબ! ગ્લોબલ ડેટાને નીચેના બધા કમ્પોનન્ટ્સમાં પ્રદાન કરવા માટે

Context.Provider ટેગનો ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ રિએક્ટના સૌથી ધમાકેદાર અને રિયલ-વર્લ્ડ પ્રોજેક્ટ્સ માટે અનિવાર્ય પ્રકરણ તરફ:

Part 4: Chapter 15 — React Router DOM. આ ચેપ્ટરમાં આપણે શીખીશું કે કેવી રીતે વેબસાઇટમાં પેજને સેકન્ડના હજારમા ભાગમાં, **બિલકુલ રીફ્રેશ (Reload) કર્યા વગર** પલક ઝપકાવતા નવું પેજ (/about, /contact) ખોલી શકાય છે, જેને સત્તાવાર રીતે **SPA (Single Page Application)** કહેવાય છે.

Part 4: Chapter 15 — React Router DOM (મોર્ડન નેવિગેશન આર્કિટેક્ચર)

1. SPA નેવિગેશન એટલે શું? (તેની જરૂર કેમ છે?)

- **જૂનો પ્રોબ્લેમ:** સાદા HTML માં આપણે જ્યારે એન્કર ટેગ () પર ક્લિક કરીએ, ત્યારે બ્રાઉઝર આખું નવું પેજ સર્વર પરથી ડાઉનલોડ કરે છે. આનાથી સ્ક્રીન વ્હાઇટ થઈને આખી વેબસાઇટ ફરીથી લોડ થાય છે, જેમાં ટાઇમ બગડે છે.
- **આધુનિક સોલ્યુશન:** રિએક્ટમાં આ પેજ રીફ્રેશ થવાની આખી સિસ્ટમ જ ગાયબ કરી દેવામાં આવી છે. આ માટે **react-router-dom** નામનું ઓફિશિયલ પેકેજ વપરાય છે. તે બ્રાઉઝરના URL રસ્તા પર નજર રાખે છે, અને યુઝર જેવી કોઈ લિંક દબાવે એટલે આખું પેજ રીલોડ કરવાના બદલે માત્ર તે બદલાયેલો કમ્પોનન્ટ લાઇવ સ્ક્રીન પર મૂકી દે છે!

2. Routing પેકેજ પ્રોજેક્ટમાં ઇન્સ્ટોલ કરવું

તમારા VS Code ના ટર્મિનલમાં આ કમાન્ડ લખીને Enter કરો જેથી આ પાવરફુલ મોર્ડન રૂટીંગ લાઇબ્રેરી પ્રોજેક્ટમાં આવી જાય:

```
bash
```

```
npm install react-router-dom
```

Use code with caution.

3. પ્રેક્ટિકલ કોડ ઉદાહરણ (સંપૂર્ણ મલ્ટી-પેજ SPA પ્રોજેક્ટ)

આપણે આખા રાઉટીંગ આર્કિટેક્ચરને સરળતાથી સમજવા માટે મુખ્ય ૪ કીવર્ડ્સ વાપરીશું:

1. **BrowserRouter:** આખા પ્રોજેક્ટની બહાર લગાવવાનું મેઇન વાતાવરણ (બોક્સ).
2. **Routes:** બધી જ લિંક્સ (રસ્તાઓ) સાચવનારો લિસ્ટ કન્ટેનર.
3. **Route:** કોઈ એક સિંગલ લિંકની શરત (દા.ત. જો પાથ /about હોય તો અબાઉટ કમ્પોનન્ટ લોડ કરો).
4. **Link:** સાદા એન્કર ટેગ <a href> નો કિંગ લિંકલ, જે પેજને **રીફ્રેશ થતા અટકાવે છે**.

તમારી **src/App.jsx** ફાઇલ આખી સાફ કરી દો અને આ આખો મલ્ટી-પેજ કોડ ત્યાં પેસ્ટ કરો:

jsx

// src/App.jsx

import React from 'react';

// ૧. ઓફિશિયલ રૂટીંગ કમ્પોનન્ટ્સ ઇમ્પોર્ટ કરવા

import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {

return (

// 🚧 ૨. આખા પ્રોજેક્ટને BrowserRouter થી લોક (પેક) કરવો

<BrowserRouter>

<div style={{ fontFamily: 'Arial, sans-serif' }}>

{/* 📖 ૩. નેવિગેશન બાર (Navbar સેક્શન) */}

<nav style={{ backgroundColor: '#2d3748', padding: '15px', display: 'flex', gap: '20px' }}>

{/* 🏠 નિયમ: <a> ટેગના બદલે હંમેશા <Link to="..."> જ લખવું જેથી પેજ રીફ્રેશ ન થાય */}

<Link to="/" style={{ color: 'white', textDecoration: 'none', fontWeight: 'bold' }}> 🏠
હોમ</Link>

<Link to="/about" style={{ color: 'white', textDecoration: 'none', fontWeight: 'bold' }}> 📘
અબાઉટ</Link>

<Link to="/contact" style={{ color: 'white', textDecoration: 'none', fontWeight: 'bold' }}> 📞
કોન્ટેક્ટ</Link>

</nav>

{/* 🚦 ૪. અસલી કંટ્રોલ રૂમ: બધી જ લિંક્સ આની અંદર ચેક થશે */}

<div style={{ padding: '30px', textAlign: 'center' }}>

<Routes>

{/* path એટલે URL ની લિંક, અને element એટલે કયો કમ્પોનન્ટ ઓપન કરવો છે તે */}

<Route path="/" element={<Home />} />

<Route path="/about" element={<About />} />

<Route path="/contact" element={<Contact />} />

```

        { /* 404 વાઇલ્ડકાર્ડ રાઉટ: જો કોઈ ખોટી લિંક નાખે તો આપમેળે આ ચાલશે */ }

        <Route path="*" element={<NotFound />} />

    </Routes>

</div>

</div>

</BrowserRouter>

);

}

// =====
// 🏠 કાલ્પનિક કમ્પોનન્ટ્સ (અલગ-અલગ પેજ્સ)
// =====

function Home() {
    return <h2> 🏠 આ આપણું હોમ પેજ (મુખ્ય પેજ) છે.</h2>;
}

function About() {
    return <h2> 📖 આ આપણું અબાઉટ પેજ છે. અમે રિએક્ટ રાઉટીંગ શીખી રહ્યા છીએ.</h2>;
}

function Contact() {
    return <h2> 📞 આ કોન્ટેક્ટ પેજ છે. ઇમેઇલ: support@school.com</h2>;
}

function NotFound() {
    return <h2 style={{ color: '#e53e3e' }}> ❌ 404 એરર: આ પેજ વેબસાઇટ પર ઉપલબ્ધ નથી!</h2>;
}

export default App;

```

Use code with caution.

4. લાઇવ બ્રાઉઝર ટેસ્ટિંગ અને જાદુ

બ્રાઉઝરમાં `http://localhost:5173` લિંક ઓપન કરો અને આ અદ્યતન આર્કિટેક્ચર ચેક કરો:

- **સુપર જાદુ:** જ્યારે તમે ઉપર બનેલા કાળા નેવબારના બટનો જેમ કે **અબાઉટ** કે **કોન્ટેક્ટ** પર ક્લિક કરશો, ત્યારે બ્રાઉઝરની ઉપર રહેલું રીલોડનું બટન એક મિલિસેકન્ડ માટે પણ ગોળ ફરશે નહીં!
- તમારી વેબસાઇટ કોઈ પણ લોડિંગ લીધા વગર, પલક ઝપકાવતા જ ડાયરેક્ટ નવું પેજ સ્ક્રીન પર બદલી આપશે! આને કહેવાય પ્રોફેશનલ મોર્ડન સિંગલ પેજ એપ્લિકેશન (SPA) [w3schools javascript tutorial]. 

આપણું **Chapter 15** અહીં રૂટીંગ આર્કિટેક્ચર અને મલ્ટી-પેજ SPA ની સુપર કિંમતી માસ્ટરી સાથે કમ્પ્લીટ થાય છે.

પ્રેક્ટિકલ ટેસ્ટ માટે:

રિએક્ટ રાઉટરમાં જ્યારે આપણે નેવિગેશન લિંક બનાવવી હોય, ત્યારે બ્રાઉઝરને જબરદસ્તી રીફ્રેશ થતું રોકવા માટે જૂના સાદા એન્કર ટેગ (`<a href="...">`) ના બદલે કયો **આધુનિક રિએક્ટ કમ્પોનન્ટ ટેગ** વાપરવો અતિશય ફરજિયાત છે?

- `<a>`
- `<Link to="...">`
- `<Route path="...">`
- `<BrowserRouter>`

જો આ રોકેટ-સ્પીડ નેવિગેશન કન્સેપ્ટ તમારા મગજમાં બરાબર લોક થઈ ગયો હોય, તો મને જણાવો:

- શું આપણે હવે આગળ વધીને આપણી રિએક્ટ સફરના સૌથી કિંમતી અને આખરી સેક્શન એટલે કે **Part 5: Full-Stack Integration & Production (Chapter 17: Axios ઇન્ટીગ્રેશન — આપણા અગાઉ બનાવેલા Node.js/PostgreSQL બેકએન્ડ સર્વર સાથે રિએક્ટને લાઇવ જોડીને અસલી ડેટા લાવવો)** તેના તરફ આગળ વધીએ?

બિલકુલ સાચો જવાબ! વેબસાઇટને રીફ્રેશ થતી રોકવા માટે સાદા એન્કર ટેગની જગ્યાએ `<Link to="...">`

કમ્પોનન્ટનો ઉપયોગ થાય છે [w3schools javascript tutorial].

ચાલો હવે આગળ વધીએ આખા કોર્સના સૌથી ધમાકેદાર અને વિજયી સેક્શન તરફ: **Part 5: Full-Stack Integration & Production**. આ પ્રકરણમાં આપણે અગાઉ બનાવેલા અસલી **Node.js/PostgreSQL બેકએન્ડ સર્વર** સાથે રિએક્ટ ફ્રન્ટએન્ડને લાઇવ જોડીશું!

આજે આપણે જોઈશું **Chapter 17: Axios Integration (બેકએન્ડ અને ફ્રન્ટએન્ડનું મહામિલન)**.

🔑 1. Axios એટલે શું? (તેની જરૂર કેમ પડી?)

- **સરળ સમજૂતી:** આપણે પ્રકરણ ૧૦ માં ઇનબિલ્ટ fetch() કમાન્ડથી ઓનલાઇન ડેટા મંગાવ્યો હતો. પરંતુ રિયલ-વર્લ્ડ પ્રોજેક્શનલ પ્રોજેક્ટ્સમાં ડેટા મંગાવવા, સાઇન-અપ કરવા કે પાસવર્ડ મોકલવા માટે **Axios** નામની સુપર-પાવરફુલ થર્ડ-પાર્ટી લાઇબ્રેરી વપરાય છે.
- **તે કેમ ખાસ છે?** Axios બહુ સ્માર્ટ છે. તે આપમેળે ડેટાને JSON ઓબ્જેક્ટમાં કન્વર્ટ કરી લે છે, એરર હેન્ડલિંગ બહુ સુંદર રીતે કરે છે, અને ગ્લોબલ ટોકન્સ (JWT) હેડર્સમાં પાસ કરવા માટે બહુ જ શોર્ટ કોડ લખવાની સુવિધા આપે છે.

📦 2. Axios પેકેજ પ્રોજેક્ટમાં ઇન્સ્ટોલ કરવું

તમારા VS Code ના ટર્મિનલમાં આ કમાન્ડ લખીને Enter કરો જેથી Axios પ્રોજેક્ટમાં આવી જાય:

```
bash
```

```
npm install axios
```

Use code with caution.

🖥️ 3. પ્રેક્ટિકલ કોડ ઉદાહરણ (અસલી બેકએન્ડ સર્વર સાથે કનેક્શન)

ધારો કે આપણે એડવાન્સ બેકએન્ડ કોર્સના પ્રકરણ ૭ માં PostgreSQL સાથે જે અસલી સર્વર બનાવ્યું હતું (જે પોર્ટ 5000 પર ચાલે છે અને બધા વિદ્યાર્થીઓનું લિસ્ટ આપે છે), તેની સાથે આ રિએક્ટને જોડીએ.

તમારી **src/App.jsx** ફાઇલમાં રહેલો બધો કોડ સાફ કરી દો અને આ લાઇવ ફૂલ-સ્ટેક કોડ ત્યાં પેસ્ટ કરો:

```
jsx
```

```
// src/App.jsx
```

```
import React, { useState, useEffect } from 'react';
```

```
import axios from 'axios'; // ૧. એક્સિઓસ ઇમ્પોર્ટ કર્યું
```

```
function App() {
```

```
  const [students, setStudents] = useState([]);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState("");
```

```
  // ૨. પેજ લોડ થતાં જ આપમેળે આપણા લોકલ કે લાઇવ બેકએન્ડ સર્વરને કોલ કરવો
```

```
  useEffect(() => {
```

```
    // Axios ની મદદથી ડેટા લાવવાનું પ્રોજેક્શનલ ફંક્શન
```

```
    const fetchBackendData = async () => {
```

```

try {
  // 🚨 જાહે: .json() કરવાની જરૂર નથી, એક્સિઓસ ડાયરેક્ટ અસલી ડેટા 'res.data' માં આપશે
  // (યાદ રાખજો પ્રકરણ ૧૪ મુજબ તમારા બેકએન્ડમાં 'cors' એલાઉ હોવું જરૂરી છે)
  const res = await axios.get('http://localhost:5000/api/sql/students');

  setStudents(res.data); // બેકએન્ડમાંથી આવેલો SQL ડેટા સ્ટેટમાં સેવ કર્યો
  setLoading(false);
} catch (err) {
  console.error("CORS અથવા સર્વર એરર: ", err.message);
  setError("❌ બેકએન્ડ સર્વર સાથે કનેક્ટ થઈ શક્યું નથી! પ્લીઝ ચેક કરો કે તમારો Node.js પ્રોજેક્ટ ચાલુ છે કે નહીં.");
  setLoading(false);
}
};

fetchBackendData();
}, []);

return (
  <div style={{ padding: '30px', fontFamily: 'Arial, sans-serif', backgroundColor: '#f7fafc',
minHeight: '100vh' }}>
    <h1 style={{ color: '#2b6cb0' }}>🔗 React + Node.js + PostgreSQL Live Integration</h1>
    <p>તમારા પોતાના અસલી બેકએન્ડ સર્વર અને SQL ડેટાબેઝમાંથી આવતો લાઈવ ડેટા નીચે મુજબ છે:</p>

    <hr style={{ margin: '20px 0', borderColor: '#e2e8f0' }} />

    {/ * એરર કે લોડિંગ હોય તો કન્ડિશનલ મેસેજ બતાવવો */}
    {loading && <h3 style={{ color: '#3182ce' }}>⌚ બેકએન્ડ સર્વર સાથે લાઈવ બ્રિજ કનેક્ટ થઈ રહ્યો છે...</h3>}
    {error && <h3 style={{ color: '#e53e3e', padding: '10px', backgroundColor: '#fff5f5',
borderRadius: '5px' }}>{error}</h3>}}

```

```

{ /* 🚦 જો ડેટા સફળતાપૂર્વક આવી જાય તો લિસ્ટ લૂપમાં બતાવવો */
  {!!loading && !error && (
    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(280px, 1fr))',
      gap: '20px' }}>
      {students.length === 0 ? (
        <p>ડેટાબેઝમાં હજુ સુધી કોઈ વિદ્યાર્થીનો ડેટા નથી!</p>
      ) : (
        students.map((std) => (
          <div key={std.id} style={{ padding: '20px', backgroundColor: 'white', borderRadius:
            '10px', boxShadow: '0 4px 6px rgba(0,0,0,0.05)', borderLeft: '5px solid #3182ce' }}>
            <h3 style={{ margin: '0 0 10px 0', color: '#2d3748' }}> 👤 {std.name}</h3>
            <p style={{ margin: '5px 0', color: '#4a5568' }}><b> 📧 ઈમેલ:</b> {std.email}</p>
            <p style={{ margin: '5px 0', color: '#4a5568' }}><b> 🎂 ઉંમર:</b> {std.age} વર્ષ</p>
            <span style={{ fontSize: '12px', color: '#a0aec0' }}>Student ID: #{std.id}</span>
          </div>
        ))
      )}
    </div>
  )};
}

```

export default App;

Use code with caution.

🚦 4. અસલી Full-Stack લાઇવ ટેસ્ટિંગ

આ જાદુ જોવા માટે તમારા કમ્પ્યુટરમાં આ આખી ચેઇન એકસાથે ચાલુ હોવી જોઈએ:

1. તમારો PostgreSQL ડેટાબેઝ બેકગ્રાઉન્ડમાં લાઇવ ચાલુ કરો.
2. તમારું Node.js/Express સર્વર (<http://localhost:5000>) બીજા ટર્મિનલમાં રન કરો.

3. તમારો આ React.js પ્રોજેક્ટ (<http://localhost:5173>) રન કરો.

- **અદ્ભુત જાદુ:** જેવું રિએક્ટનું પેજ ખુલશે, તે આપમેળે પાછળના રસ્તેથી એક્સિઓસ દ્વારા તમારા Node.js સર્વર પર જશે, સર્વર ડેટાબેઝમાંથી વિદ્યાર્થીઓનો સાચો રેકૉર્ડ બહાર કાઢશે, અને રિએક્ટ તેને આપમેળે સ્ક્રીન પર સુંદર કાર્ડ્સ બનાવીને સજાવી દેશે! 🎉

🎉 **હાર્દિક અભિનંદન!** આપણું **Part 5: Full-Stack Integration (Chapter 17)** અહીં પૂરેપૂરું અને અદ્ભુત રીતે કમ્પ્લીટ થાય છે. તમે હવે કોઈ સામાન્ય ફ્રન્ટએન્ડ ડેવલપર નથી, પણ ફ્રન્ટએન્ડ અને બેકએન્ડના તાર એકસાથે લાઈવ જોડવાના અસલી **માસ્ટર Full-Stack Developer** બની ગયા છો!